

Server-Side Testing

[Jest](#) · [Supertest](#) · [Socket.io](#) · [Prisma](#) · [k6](#)

Author: Daniel Boone

Repository: github.com/dboone323/HavenWorld (GPL-3.0)

Date: Friday, 18 September 2026

Environment: [MAC] MacBook Pro local dev | [LINUX] Oracle Cloud ARM VM (production)

Stack: Node.js 22 LTS · Express 5 · Socket.io 4 · Prisma ORM · PostgreSQL 16 · Jest 29 · k6

Cost Constraint: 100% free-tier services only

Prerequisites

Before beginning Part 7A, the following parts must be completed:

- **Part 1** — Oracle Cloud VM provisioning, SSH keys, firewall rules
- **Part 2** — Node.js 22, PostgreSQL 16, Redis 7 installation on Oracle VM
- **Part 3** — Express 5 server scaffold, Prisma schema & migrations
- **Part 4** — Authentication system (JWT + refresh tokens)
- **Part 5** — Socket.io 4 room/event architecture
- **Part 6** — Economy, Inventory, Workshop, and Trade services

All commands are run either on the **[MAC]** MacBook Pro or the **[LINUX]** Oracle Cloud VM. Labels appear before every command block.

Table of Contents

1. Introduction & Testing Philosophy

- 1.1 Why game servers need deeper testing
- 1.2 Testing pyramid for HavenWorld
- 1.3 Test file structure & naming conventions
- 1.4 Tools summary table

2. Installing & Configuring Jest [MAC]

- 2.1 npm dependencies
- 2.2 jest.config.ts
- 2.3 tsconfig.test.json
- 2.4 package.json scripts
- 2.5 .env.test file
- 2.6 GitHub Actions PR check

3. Test Database Setup [LINUX]

- 3.1 Create havenworld_test database
- 3.2 Run migrations on test DB
- 3.3 Global setup & teardown files
- 3.4 Factory helpers
- 3.5 Transaction rollback pattern

4. Unit Tests — Auth Middleware [MAC]

5. Unit Tests — ModerationService [MAC]

6. Unit Tests — RoomManager [MAC]

7. Unit Tests — InventoryService & Economy [MAC]

8. Unit Tests — WorkshopService & CraftingQueue [MAC]

9. Unit Tests — TradeManager (P2P Trade) [MAC]

10. Integration Tests — Full Auth Flow [MAC]

11. Integration Tests — Socket.io Connection & Room Flow [MAC]

12. Integration Tests — Fishing Mini-Game Server Validation [MAC]

13. Integration Tests — Loft Privacy & Doorbell [MAC]

14. Database Tests — Prisma Integration [LINUX]

15. Load Tests — k6 Scripts [LINUX]

16. CI Integration — GitHub Actions [MAC]

17. Test Utilities & Helpers [MAC]

18. Test Coverage Checklist

1. Introduction & Testing Philosophy

1.1 Why Game Servers Need Deeper Testing Than Typical REST APIs

A conventional REST API is largely *stateless*: each request carries all context it needs, and responses are independent. HavenWorld's server is fundamentally different. Rooms are live, in-memory graphs of connected sockets. The economy is a shared ledger where concurrent writes can produce phantom coins. Real-time events must be delivered in the right sequence to the right subset of clients. A bug in any of these systems does not merely return a wrong value — it corrupts the shared world state for every player currently online.

Consider the failure modes unique to a social virtual world:

- **Race conditions in coin transfers:** Two simultaneous trade confirmations could double-credit a player's balance if the Prisma transaction is not atomic.
- **Event fan-out correctness:** A PLAYER_MOVED broadcast must reach every socket in the same room *except* the sender. An off-by-one or missing filter causes ghost movements or self-echo loops.
- **Anti-cheat bypass:** A client sending manipulated PLAYER_MOVE deltas can teleport across the map. Server-side validation must reject any delta exceeding the maximum physics speed.
- **Economy exploits:** Negative coin transfers, self-transfers, duplicate fishing rewards, and earnings cap overflows all require explicit test coverage — a linter will never catch them.
- **Room lifecycle leaks:** An empty room that is never cleaned up from memory will slowly exhaust the server's heap during a long session. The RoomManager must be tested to confirm cleanup fires when the last user leaves.

These failure modes demand a test suite that goes well beyond `expect(200)` status code assertions. Every service that touches state — rooms, economy, inventory, moderation, crafting, trading — must have unit tests that isolate it, integration tests that prove it works end-to-end, and load tests that confirm it holds up under concurrent pressure.

1.2 Testing Pyramid for HavenWorld

HavenWorld follows a classic test pyramid with four layers, each building confidence on top of the last:

Layer	Tool	Scope	Run On	Speed	Count Target
Unit	Jest 29 + ts-jest	Single service/function, all deps mocked	[MAC] locally, CI on PR	Very fast (<5 s total)	80-120 tests
Integration — HTTP	Jest + Supertest	Full Express route → DB round-trip	[MAC] locally, CI on PR	Moderate (10-30 s)	30-50 tests
Integration — WebSocket	Jest + socket.io-client	In-process Socket.io server + real events	[MAC] locally, CI on PR	Moderate (15-45 s)	20-35 tests
Load	k6 (open-source)	Concurrent users, thresholds on p95 latency	[LINUX] Oracle VM, nightly	Slow (1-5 min)	3-5 scripts

Unit tests give fast, precise feedback. Integration tests verify that the layers cooperate correctly. Load tests confirm the system is production-worthy before every nightly deploy window. The pyramid shape is intentional: many cheap unit tests, fewer expensive integration tests, a handful of targeted load tests.

1.3 Test File Structure & Naming Conventions

Tests live either co-located with source files (unit tests) or in a top-level `__tests__` directory (integration and database tests). This structure makes it easy to find the test for any module and keeps CI configuration simple.

```

HavenWorld/ |─ src/ |─ server/ |─ middleware/ |─ |─ |─
__tests__/ |─ |─ |─ auth.test.ts |─ unit test co-located
with source |─ |─ services/ |─ __tests__/ |─
|─ moderation.test.ts |─ |─ roomManager.test.ts |─
|─ inventoryService.test.ts |─ |─ workshopService.test.ts |─
|─ tradeManager.test.ts |─ |─ tests/ |─ integration/ |─ |─
auth.integration.test.ts |─ |─ |─ socketio.integration.test.ts |─ |─ |─
fishing.integration.test.ts |─ |─ |─ loft.integration.test.ts |─ |─ |─
database/ |─ |─ |─ prisma.integration.test.ts |─ |─ helpers/ |─ |─
factories.ts |─ |─ |─ socketHelpers.ts |─ |─ dbHelpers.ts |─ |─
|─ jwtHelpers.ts |─ |─ load-tests/ |─ |─ ws-concurrent.js |─ |─ api-
auth.js |─ |─ fishing-concurrent.js |─ |─ jest.config.ts |─ |─
tsconfig.test.json |─ |─ .env.test

```

Naming conventions:

- Unit tests: `[module].test.ts`
- Integration tests: `[feature].integration.test.ts`
- Database tests: `[scope].integration.test.ts` inside `__tests__/database/`
- Load test scripts: `[scenario].js` inside `load-tests/` (k6 uses plain JS, not TypeScript)
- All describe blocks use the service name: `describe('AuthMiddleware', ...)`
- All it/test strings use should-language: `it('should return 401 when token is expired', ...)`

1.4 Tools Summary Table

Tool	Version	Purpose	License	Cost
Jest	29.x	Test runner, assertions, mocking, coverage	MIT	Free
ts-jest	29.x	TypeScript transformer for Jest (no separate build step)	MIT	Free
Supertest	7.x	HTTP assertions against Express app without starting a live server	MIT	Free
socket.io-client	4.x	Real WebSocket	MIT	Free

Tool	Version	Purpose	License	Cost
		client for in-process Socket.io integration tests		
k6	latest OSS	Load & performance testing; threshold assertions on p95 latency	AGPL-3.0	Free (OSS binary)
@types/jest	29.x	TypeScript type definitions for Jest globals	MIT	Free
@types/supertest	6.x	TypeScript type definitions for Supertest	MIT	Free

2. Installing & Configuring Jest MAC

2.1 Install npm Dependencies

Run the following from the root of the HavenWorld repository on your MacBook Pro. These are all `devDependencies` — they are never bundled into production.

MAC

```
npm install --save-dev \  jest@29 \  ts-jest@29 \  @types/jest@29 \  
supertest@7 \  @types/supertest@6 \  socket.io-client@4
```

Why ts-jest and not babel-jest?

ts-jest performs full TypeScript type checking during test runs. This means a type error in a service method will fail the test suite immediately, not silently produce any casts. For a game economy where type safety prevents coin duplication bugs, this trade-off in compile speed is worth it.

2.2 jest.config.ts

Create this file at the repository root. The `ts-jest` preset handles all TypeScript compilation. `forceExit: true` is critical for Socket.io tests — without it, open socket handles prevent Jest from cleanly exiting.

MAC

```
// jest.config.ts import type { Config } from 'jest'; const config: Config =
{ // Use ts-jest preset so TypeScript files are compiled on-the-fly
  preset: 'ts-jest', // Node environment (not jsdom – this is a server-side
  test suite) testEnvironment: 'node', // Match all .test.ts
  and .integration.test.ts files in any __tests__ dir testMatch: [
    '**/__tests__/**/*.test.ts', '**/__tests__/**/*.integration.test.ts',
    '**/src/**/*.test.ts', ], // Give each test file up to 15
  seconds – Socket.io handshakes need headroom testTimeout: 15000, //
  Force Jest to exit after all tests complete (critical for open socket
  handles) forceExit: true, // Detect resource leaks (open handles,
  pending timers) and warn about them detectOpenHandles: true, // Global
  setup: runs ONCE before all test suites – migrations + seed globalSetup:
  '/src/server/tests/jest.globalSetup.ts', // Global teardown: runs ONCE
  after all test suites – truncate tables globalTeardown:
  '/src/server/tests/jest.globalTeardown.ts', // Coverage configuration
  collectCoverageFrom: [ 'src/server/**/*.ts', '!src/server/**/*.d.ts',
    '!src/server/types/**', '!src/server/generated/**', // Exclude Prisma
    generated client ], coverageThresholds: { global: { branches:
    60, functions: 60, lines: 60, statements: 60, }, },
  coverageReporters: ['text', 'lcov', 'html'], coverageDirectory: 'coverage',
  // Load .env.test before any test runs setupFiles:
  ['/src/server/tests/loadEnv.ts'], // Project-level setup: reset mocks
  between tests setupFilesAfterFramework: [], // TypeScript path aliases
  (must mirror tsconfig.json paths) moduleNameMapper: { '@server/(.*)$':
  '/src/server/$1', '@tests/(.*)$': '/__tests__$1', }, // Separate
  test projects for granular npm scripts projects: [ { displayName:
  'unit', testMatch: ['**/src/**/*.test.ts'], preset:
  'ts-jest', testEnvironment: 'node', }, { displayName:
  'integration', testMatch: ['**/__tests__/integration/**/*.test.ts',
  '**/__tests__/database/**/*.test.ts'], preset: 'ts-jest',
  testEnvironment: 'node', }, ], }; export default config;
```

2.3 tsconfig.test.json

A separate TypeScript configuration for tests allows including test helper files and relaxing some strictness rules without polluting the production build configuration.

MAC

```
// tsconfig.test.json – extends main tsconfig, adds test-specific includes {
  "extends": "./tsconfig.json", "compilerOptions": { // Allow default
  imports from modules without a default export (common in test helpers)
  "esModuleInterop": true, // Include @types/jest globals (describe, it,
  expect, jest.fn, etc.) "types": ["jest", "node"], // Source maps for
  readable stack traces in test output "sourceMap": true,
  "inlineSourceMap": false }, "include": [ "src/**/*.ts",
  "__tests__/**/*.ts", "jest.config.ts", "jest.globalSetup.ts",
```

```
"jest.globalTeardown.ts" ], "exclude": [ "node_modules", "dist",  
"load-tests" // k6 scripts are plain JS, not TypeScript ] }
```

2.4 package.json Scripts

Add these scripts to `package.json`. Each target maps to a specific Jest configuration or project filter.

MAC

```
// package.json (scripts section – partial) { "scripts": { // Run all  
tests (unit + integration) once "test": "jest --config jest.config.ts",  
// Run unit tests only (fast – no DB required) "test:unit": "jest --  
config jest.config.ts --selectProjects unit", // Run integration tests  
only (requires havenworld_test DB) "test:integration": "jest --config  
jest.config.ts --selectProjects integration", // Run all tests with  
coverage report (used in CI) "test:coverage": "jest --config  
jest.config.ts --coverage", // Interactive watch mode – reruns tests on  
file save (great for TDD) "test:watch": "jest --config jest.config.ts --  
watch", // Run a single test file by path "test:file": "jest --  
config jest.config.ts --testPathPattern" } }
```

2.5 .env.test File

Create `.env.test` at the repository root. This file is loaded by `setupFiles` before any test runs. **Never commit this file to git.** Add it to `.gitignore` and share it with teammates via a secrets manager.

MAC

```
# .env.test – Test environment variables # NEVER commit this file. Add  
to .gitignore. # Points to the ISOLATED test database – never  
havenworld_prod  
DATABASE_URL="postgresql://havenworld_user:your_password@localhost:5432/  
havenworld_test" # Separate JWT secret for tests – prevents test tokens from  
being valid in production  
JWT_SECRET="test_jwt_secret_not_for_production_use_only"  
JWT_REFRESH_SECRET="test_refresh_secret_not_for_production_use_only"  
JWT_EXPIRY="15m" JWT_REFRESH_EXPIRY="7d" # Redis for rate limiter / doorbell  
tests REDIS_URL="redis://localhost:6379/1" # DB index 1 isolates test data  
from dev (index 0) # Email provider – use a no-op stub in tests, not real  
Resend API RESEND_API_KEY="test_resend_key_noop"  
EMAIL_FROM="test@havenworld.local" # App NODE_ENV="test" PORT=3001 #  
Different port from dev server (3000) to allow parallel running  
LOG_LEVEL="error" # Suppress info/debug logs during test runs for cleaner  
output
```

Important: DATABASE_URL Must Point to havenworld_test

If `DATABASE_URL` in `.env.test` ever points to `havenworld_prod`, the global teardown's `TRUNCATE` cascade will wipe your production data. Double-check the

database name every time you set up a new development machine. The `globalSetup` file (Section 3.3) includes a guard that throws if `DATABASE_URL` does not contain the string `"_test"`.

2.6 GitHub Actions PR Check (Preview)

The full GitHub Actions YAML is covered in Section 16. The key principle: every pull request to `main` must pass the full Jest suite with coverage above 60% before it can be merged. No exceptions.

3. Test Database Setup LINUX

3.1 Create the `havenworld_test` Database

SSH into the Oracle Cloud VM. The test database must be completely isolated from the production database. It shares the same PostgreSQL instance but is a separate database with its own schema, so migrations can be run and tables truncated without any risk to production data.

LINUX

```
# Connect to PostgreSQL as the superuser sudo -u postgres psql -- Create the
isolated test database CREATE DATABASE havenworld_test; -- Grant all
privileges to the app user (same user as production for simplicity) GRANT ALL
PRIVILEGES ON DATABASE havenworld_test TO havenworld_user; -- Verify both
databases exist \l -- Exit psql \q
```

3.2 Run Migrations on the Test Database

Prisma's `migrate deploy` applies all pending migration files without generating new ones. This is safe to run on both production and test databases.

LINUX

```
# Navigate to the project root on the Oracle VM cd
/home/havenworld/HavenWorld # Set DATABASE_URL to the test database for this
command only
DATABASE_URL="postgresql://havenworld_user:your_password@localhost:5432/
havenworld_test" \ npx prisma migrate deploy --schema=prisma/schema.prisma
# Verify tables were created (should see 25+ tables)
DATABASE_URL="postgresql://havenworld_user:your_password@localhost:5432/
```

```

havenworld_test" \ npx prisma db execute --stdin < 3.3 Global Setup and
Teardown Files Jest's globalSetup runs once in a separate Node process
before any test suite begins. globalTeardown runs once after all suites
complete. These are the right place for expensive operations like migrations
and truncation – not inside individual test files. jest.globalSetup.ts
// src/server/tests/jest.globalSetup.ts import { execSync } from
'child_process'; import * as dotenv from 'dotenv'; export default async
function globalSetup() { // Load test environment variables
dotenv.config({ path: '.env.test' }); const dbUrl =
process.env.DATABASE_URL; // SAFETY GUARD: Refuse to run if DATABASE_URL
doesn't reference the test DB. // This prevents accidental wipes of
production data. if (!dbUrl || !dbUrl.includes('_test')) { throw new
Error(`SAFETY: DATABASE_URL does not contain "_test".` + `Got:
${dbUrl}. Refusing to run global setup.`); }
console.log('[globalSetup] Running Prisma migrations on test database...');
// Apply any pending migrations to the test DB execSync('npx prisma migrate
deploy --schema=prisma/schema.prisma', { env: { ...process.env,
DATABASE_URL: dbUrl }, stdio: 'inherit' });
console.log('[globalSetup] Seeding minimal test data...'); // Import and
run the minimal seed (creates default rooms, item catalog) const
{ seedMinimalData } = await import('./__tests__/helpers/dbHelpers'); await
seedMinimalData(); console.log('[globalSetup] Complete. Test suite
ready.');
```

```

} jest.globalTeardown.ts
// src/server/tests/jest.globalTeardown.ts import * as dotenv from 'dotenv';
export default async function globalTeardown() { dotenv.config({ path:
'.env.test' }); // Import helper only after env is loaded const
{ truncateAllTables } = await import('./__tests__/helpers/dbHelpers');
console.log('[globalTeardown] Truncating all test database tables...'); //
TRUNCATE (not DROP) – preserves schema and migration history. // RESTART
IDENTITY resets auto-increment sequences. // CASCADE handles FK-dependent
tables automatically. await truncateAllTables();
console.log('[globalTeardown] Complete. Test database cleaned.');
```

3.4 Factory Helper Functions

Factory helpers create consistent, minimal test data. They accept optional overrides so each test can customize only what it cares about. See Section 17 for the full factories.ts implementation. //

```

__tests__/helpers/factories.ts (signature preview) // Creates a user with a
verified email and a hashed password export async function
createTestUser(overrides?: Partial<UserCreateInput>): Promise<User> //
Creates a public room owned by a given userId export async function
createTestRoom(ownerId: string, overrides?: Partial<RoomCreateInput>):
Promise<Room> // Creates an item in the item catalog (not in a user's
inventory) export async function createTestItem(overrides?:
Partial<ItemCreateInput>): Promise<Item> // Creates an invite code (unused
by default) export async function createTestInviteCode(overrides?:
Partial<InviteCodeCreateInput>): Promise<InviteCode>
```

3.5 Transaction Rollback Pattern

For integration tests that write to the database, wrap each test in a Prisma interactive transaction that always rolls back. This keeps tests fully isolated without requiring a full truncation between tests. //

Pattern: wrap test body in a transaction that always rolls back. // Prisma does not natively support savepoints, so we use \$transaction // with a manual ROLLBACK via a thrown error.

```

import { prisma } from '@server/lib/prisma';
describe('SomeIntegrationTest', () => { let tx: Prisma.TransactionClient;
beforeEach(async () => { // Start a transaction – all writes inside will
be rolled back in afterEach // Note: $transaction with an interactive
callback keeps the connection open // until the Promise resolves or
rejects. // We store the client in `tx` so tests can use it. tx =
await (prisma as any).$begin(); // or use pg directly for SAVEPOINT support
}); afterEach(async () => { // Always roll back, regardless of test
outcome await (prisma as any).$rollback(); }); it('should create a
user without persisting to the real DB', async () => { const user = await
```

```

tx.user.create({      data: { email: 'temp@test.com', username: 'temp',
passwordHash: 'x' },      });      expect(user.id).toBeDefined();      // After
this test, the user row is rolled back – next test sees a clean DB      }); });
📝 Note: Rollback vs. Truncation      The rollback pattern is fastest for
integration tests because it avoids the overhead of re-seeding after every
test. The global teardown truncation is a final safety net, not the primary
isolation mechanism. Use rollback inside test files; use truncation only in
globalTeardown.      4. Unit Tests – Auth Middleware MAC      File:
src/server/middleware/__tests__/auth.test.ts      The auth middleware is the
gateway to every protected route. A bypass here grants unauthenticated access
to inventory, economy, and room management. These tests use jest.mock to stub
Prisma – no database connection is needed, keeping the suite fast.      //
src/server/middleware/__tests__/auth.test.ts      import { Request, Response,
NextFunction } from 'express';      import jwt from 'jsonwebtoken';      import
{ authMiddleware } from '../auth';      import { prisma } from
'@server/lib/prisma';      import { generateTestToken, generateExpiredToken } from
'@tests/helpers/jwtHelpers';      // Mock Prisma at the module level – unit tests
must NOT hit a real database.      // This makes these tests runnable with no DB
connection at all.      jest.mock('@server/lib/prisma', () => ({      prisma:
{      user: {      findUnique: jest.fn(),      },      },      }));      const
mockPrismaUser = prisma.user.findUnique as jest.MockedFunction<typeof
prisma.user.findUnique>;      // Helper: create mock Express
request/response/next function      mockReq(authHeader?: string): Partial<Request>
{      return {      headers: authHeader ? { authorization: authHeader } :
{},      };      };      function mockRes(): { status: jest.Mock; json: jest.Mock; res:
Partial<Response> } {      const json = jest.fn();      const status =
jest.fn().mockReturnValue({ json });      return { status, json, res: { status }
as any };      };      describe('AuthMiddleware', () => {      let next: NextFunction;
beforeEach(() => {      next = jest.fn();
jest.clearAllMocks();      });      //
// a) Valid JWT → sets
req.user correctly      // -----
it('should set req.user when a valid JWT is provided', async () =>
{      const userId = 'user_abc123';      const token =
generateTestToken(userId);      // Simulate Prisma finding the user in the
database      mockPrismaUser.mockResolvedValue({ id: userId, email:
'test@example.com', username: 'tester', isBanned: false } as any);      const
req = mockReq(`Bearer ${token}`) as any;      const { res } = mockRes();
await authMiddleware(req, res as any, next);      // Middleware must attach
the user object for downstream route handlers
expect(req.user).toEqual(expect.objectContaining({ id: userId }));
expect(next).toHaveBeenCalledTimes(1);
expect(next).toHaveBeenCalledWith(); // Called with no error      });      //
// b) Missing Authorization
header → 401      // -----
it('should
return 401 when Authorization header is absent', async () => {      const req
= mockReq(undefined) as any; // No header      const { status, json, res } =
mockRes();      await authMiddleware(req, res as any, next);
expect(status).toHaveBeenCalledWith(401);
expect(json).toHaveBeenCalledWith(expect.objectContaining({ error:
expect.stringContaining('token') }));
expect(next).not.toHaveBeenCalled();      });      //
// c) Malformed token → 401
// -----
it('should return 401 when
token is a garbage string', async () => {      const req = mockReq('Bearer
not.a.real.jwt.garbage') as any;      const { status, json, res } = mockRes();
await authMiddleware(req, res as any, next);
expect(status).toHaveBeenCalledWith(401);
expect(json).toHaveBeenCalledWith(expect.objectContaining({ error:
expect.any(String) }));      expect(next).not.toHaveBeenCalled();      });      //

```

```

// d) Expired token → 401
with "Token expired" message //
it('should return 401 with
"Token expired" message when JWT is expired', async () => { const token =
generateExpiredToken('user_xyz'); const req = mockReq(`Bearer ${token}`)
as any; const { status, json, res } = mockRes(); await
authMiddleware(req, res as any, next);
expect(status).toHaveBeenCalledWith(401); // The client uses this
specific string to trigger a token refresh flow
expect(json).toHaveBeenCalledWith(expect.objectContaining({ error: 'Token
expired' })); expect(next).not.toHaveBeenCalled(); }); //
// e) Token signed with wrong
secret → 401 // it('should
return 401 when token is signed with the wrong secret', async () => { //
Sign with a different secret than what the middleware uses const badToken
= jwt.sign({ userId: 'user_abc' }, 'wrong_secret_entirely', { expiresIn:
'15m' }); const req = mockReq(`Bearer ${badToken}`) as any; const
{ status, json, res } = mockRes(); await authMiddleware(req, res as any,
next); expect(status).toHaveBeenCalledWith(401);
expect(next).not.toHaveBeenCalled(); }); //
// f) Valid token but user
not in DB → 401 //
it('should return 401 when token is valid but user does not exist in
database', async () => { const token =
generateTestToken('deleted_user_id'); // Simulate user having been
deleted after token was issued mockPrismaUser.mockResolvedValue(null);
const req = mockReq(`Bearer ${token}`) as any; const { status, json,
res } = mockRes(); await authMiddleware(req, res as any, next); //
Token is cryptographically valid, but the user no longer exists – must
reject. // This prevents deleted/banned accounts from continuing to use
pre-issued tokens. expect(status).toHaveBeenCalledWith(401);
expect(next).not.toHaveBeenCalled(); }); });
5. Unit Tests –
ModerationService MAC File: src/server/services/__tests__/moderation.test.ts
Chat moderation is safety-critical. A missed profanity filter or a broken
rate limiter exposes players (including minors) to abuse. All Redis
interactions are mocked so rate limiter logic can be tested without a running
Redis server in unit test mode. //
src/server/services/__tests__/moderation.test.ts import { ModerationService }
from '../moderationService'; // Mock Redis client – unit tests must not
require a running Redis instance. // We mock the exact methods our rate
limiter calls. jest.mock('@server/lib/redis', () => ({ redis: { incr:
jest.fn(), expire: jest.fn(), get: jest.fn(), }, })); import
{ redis } from '@server/lib/redis'; const mockIncr = redis.incr as
jest.Mock; const mockExpire = redis.expire as jest.Mock; const mockGet =
redis.get as jest.Mock; describe('ModerationService', () => { let mod:
ModerationService; beforeEach(() => { mod = new ModerationService();
jest.clearAllMocks(); }); //
// a) Clean message → allowed
// it('should allow a chat
message with no profanity', async () => { const result = await
mod.checkMessage({ userId: 'u1', message: 'Hello everyone!', isMuted: false,
isAdmin: false }); expect(result.allowed).toBe(true);
expect(result.sanitized).toBe('Hello everyone!'); }); //
// b) Profanity → blocked,
sanitized version returned //
it('should block a message containing profanity and return sanitized
version', async () => { const result = await
mod.checkMessage({ userId: 'u1', message: 'You are such a
[profanity] player!', isMuted: false, isAdmin: false, });
expect(result.allowed).toBe(false); // The sanitized version replaces bad

```

```

words with asterisks for logging purposes
expect(result.sanitized).toContain('***'); }); //
// c) Message over 200
characters → rejected //
it('should reject a message that exceeds the 200 character limit', async ()
=> { const longMessage = 'a'.repeat(201); const result = await
mod.checkMessage({ userId: 'u1', message: longMessage, isMuted: false,
isAdmin: false }); expect(result.allowed).toBe(false);
expect(result.reason).toBe('MESSAGE_TOO_LONG'); }); //
// d) Rate limiter: 6th
message in 3 seconds → blocked //
it('should block the 6th
message when 5 have been sent within 3 seconds', async () => { // First 5
messages: incr returns 1 through 5 → all allowed
mockIncr .mockResolvedValueOnce(1) .mockResolvedValueOnce(2)
.mockResolvedValueOnce(3) .mockResolvedValueOnce(4) .mockResolved
ValueOnce(5) .mockResolvedValueOnce(6); // 6th → over limit for
(let i = 0; i < 5; i++) { const r = await mod.checkMessage({ userId:
'u1', message: 'Hi', isMuted: false, isAdmin: false });
expect(r.allowed).toBe(true); } const blocked = await
mod.checkMessage({ userId: 'u1', message: 'Hi', isMuted: false, isAdmin:
false }); expect(blocked.allowed).toBe(false);
expect(blocked.reason).toBe('RATE_LIMITED'); }); //
// e) Rate limiter: after
cooldown → message allowed //
it('should allow a message after the 3-second rate limit window expires',
async () => { // Simulate Redis key expiring – incr returns 1 (fresh
window) mockIncr.mockResolvedValue(1);
mockExpire.mockResolvedValue(1); const result = await mod.checkMessage({
userId: 'u1', message: 'Back again!', isMuted: false, isAdmin: false });
expect(result.allowed).toBe(true); }); //
// f) Muted user → all
messages blocked //
it('should block all messages from a muted user', async () => { const
result = await mod.checkMessage({ userId: 'u1', message: 'Hello!', isMuted:
true, isAdmin: false }); expect(result.allowed).toBe(false);
expect(result.reason).toBe('USER_MUTED'); // Verify Redis incr was NOT
called – muted users skip the rate limiter entirely
expect(mockIncr).not.toHaveBeenCalled(); }); //
// g) Admin user → bypasses
profanity filter //
it('should allow admin users to send messages that would fail the profanity
filter', async () => { // Admins may need to quote reported messages in
moderation channels const result = await mod.checkMessage({ userId:
'admin1', message: 'Reviewing report: [profanity] content flagged',
isMuted: false, isAdmin: true, });
expect(result.allowed).toBe(true); // Message is passed through
unmodified for admin expect(result.sanitized).toBe('Reviewing report:
[profanity] content flagged'); }); //
// h) Empty string → rejected
it('should reject an empty
string message', async () => { const result = await
mod.checkMessage({ userId: 'u1', message: '', isMuted: false, isAdmin:
false }); expect(result.allowed).toBe(false);
expect(result.reason).toBe('EMPTY_MESSAGE'); }); });

```

6. Unit Tests – RoomManager

MAC File: src/server/services/__tests__/roomManager.test.ts

The RoomManager holds in-memory state for every active room. Bugs here affect all players simultaneously. The anti-cheat movement validation test is especially important – without it, a modified client can send large position deltas to teleport.

// src/server/services/__tests__/roomManager.test.ts import

```

{ RoomManager } from '../roomManager'; import { EventEmitter } from 'events';
// Mock the Socket object – we only need emit() and id for these tests
function createMockSocket(socketId: string): any { const emitter = new
EventEmitter(); return { id: socketId, emit: jest.fn(), to:
jest.fn().mockReturnThis(), broadcast: { to: jest.fn().mockReturnThis(),
emit: jest.fn() }, }; } describe('RoomManager', () => { let roomManager:
RoomManager; beforeEach(() => { // Fresh RoomManager instance per test
– no shared state between tests roomManager = new RoomManager(); });
// _____ // a) User joins room →
room.users map updated, user receives ROOM_STATE //
it('should add user to room
and emit ROOM_STATE on join', () => { const socket =
createMockSocket('socket1'); const userId = 'user_1';
roomManager.joinRoom({ socket, userId, roomId: 'town_square', username:
'Alice' }); const room = roomManager.getRoom('town_square');
expect(room).not.toBeNull(); expect(room!.users.has(userId)).toBe(true);
// Client must receive the full room state to render other players
expect(socket.emit).toHaveBeenCalledWith('ROOM_STATE',
expect.objectContaining({ roomId: 'town_square', users:
expect.any(Array), })); }); //
// b) User leaves → removed
from map, others get USER_LEFT //
it('should remove user from
room and broadcast USER_LEFT to remaining users', () => { const socket1 =
createMockSocket('socket1'); const socket2 = createMockSocket('socket2');
roomManager.joinRoom({ socket: socket1, userId: 'user_1', roomId:
'town_square', username: 'Alice' }); roomManager.joinRoom({ socket:
socket2, userId: 'user_2', roomId: 'town_square', username: 'Bob' });
roomManager.leaveRoom({ socket: socket1, userId: 'user_1', roomId:
'town_square' }); const room = roomManager.getRoom('town_square');
expect(room!.users.has('user_1')).toBe(false); // Bob must be notified
so his client removes Alice's avatar
expect(socket2.emit).toHaveBeenCalledWith('USER_LEFT',
expect.objectContaining({ userId: 'user_1' })); }); //
// c) PLAYER_MOVE →
validated, broadcast to room except sender //
it('should broadcast
PLAYER_MOVED to all room members except the moving player', () => { const
socket1 = createMockSocket('socket1'); const socket2 =
createMockSocket('socket2'); roomManager.joinRoom({ socket: socket1,
userId: 'user_1', roomId: 'room_a', username: 'Alice' });
roomManager.joinRoom({ socket: socket2, userId: 'user_2', roomId: 'room_a',
username: 'Bob' }); const movePayload = { x: 105, y: 200, userId:
'user_1', roomId: 'room_a' }; roomManager.handlePlayerMove({ socket:
socket1, payload: movePayload }); // Sender must NOT receive their own
movement echo const socket1Events = (socket1.emit as
jest.Mock).mock.calls.map(c => c[0]);
expect(socket1Events).not.toContain('PLAYER_MOVED'); // Bob must receive
Alice's new position to update her avatar
expect(socket2.emit).toHaveBeenCalledWith('PLAYER_MOVED',
expect.objectContaining({ userId: 'user_1', x: 105 })); }); //
// d) Movement speed
validation – anti-cheat //
it('should reject a PLAYER_MOVE where position delta exceeds max speed (anti-
cheat)', () => { const socket1 = createMockSocket('socket1');
roomManager.joinRoom({ socket: socket1, userId: 'user_1', roomId: 'room_a',
username: 'Alice', startX: 100, startY: 100 }); // Delta of 999 pixels
in one tick is physically impossible at normal walk speed const
cheatingMove = { x: 1099, y: 100, userId: 'user_1', roomId: 'room_a' };
const result = roomManager.handlePlayerMove({ socket: socket1, payload:

```

```

cheatingMove }); expect(result.accepted).toBe(false);
expect(result.reason).toBe('SPEED_VIOLATION'); // Optionally, the server
may emit a warning back to the cheating socket
expect(socket1.emit).toHaveBeenCalledWith('MOVE_REJECTED',
expect.objectContaining({ reason: 'SPEED_VIOLATION' })); }); //
// e) Room at capacity (20) →
21st join rejected // -----
it('should reject a 21st user when room capacity is 20', () => { const
roomId = 'small_room'; // Fill the room to capacity for (let i = 1;
i <= 20; i++) { const s = createMockSocket(`socket${i}`);
roomManager.joinRoom({ socket: s, userId: `user_${i}`, roomId, username:
`User${i}` }); } const lateSocket = createMockSocket('socket21');
const result = roomManager.joinRoom({ socket: lateSocket, userId: 'user_21',
roomId, username: 'Latecomer' }); expect(result.success).toBe(false);
expect(result.reason).toBe('ROOM_FULL');
expect(lateSocket.emit).toHaveBeenCalledWith('JOIN_FAILED',
expect.objectContaining({ reason: 'ROOM_FULL' })); }); //
// f) User moves in room they
haven't joined → rejected // -----
it('should reject PLAYER_MOVE from a user who has not joined the room', () =>
{ const socket = createMockSocket('socket1'); // Never call joinRoom
- user has no presence in room_a const result =
roomManager.handlePlayerMove({ socket, payload: { x: 100, y: 200,
userId: 'user_1', roomId: 'room_a' }, });
expect(result.accepted).toBe(false);
expect(result.reason).toBe('NOT_IN_ROOM'); }); //
// g) getRoom() for non-
existent room → creates new room // -----
it('should create and return
a new empty room when getRoom is called with an unknown roomId', () =>
{ const room = roomManager.getRoom('brand_new_room'); // Non-existent
rooms are auto-created on demand (lazy initialization)
expect(room).not.toBeNull();
expect(room!.users.size).toBe(0); }); //
// h) Room cleanup when last
user leaves // ----- it('should
remove the room from memory when the last user leaves', () => { const
socket = createMockSocket('socket1'); roomManager.joinRoom({ socket,
userId: 'user_1', roomId: 'temp_room', username: 'Solo' });
roomManager.leaveRoom({ socket, userId: 'user_1', roomId: 'temp_room' });
// Room must be deleted from the internal Map – no memory leak const
roomExists = roomManager.hasRoom('temp_room');
expect(roomExists).toBe(false); }); });
7. Unit Tests –
InventoryService & Economy MAC File:
src/server/services/__tests__/inventoryService.test.ts The economy is
HavenWorld's core engagement loop. Every test in this section directly maps
to a category of exploit that real players will attempt. Atomicity of coin
transfers is non-negotiable – a non-atomic transfer under load can produce
free coins through race conditions. //
src/server/services/__tests__/inventoryService.test.ts import
{ InventoryService } from '../inventoryService'; import { prisma } from
'@server/lib/prisma'; import { ItemNotFoundError, InsufficientFundsError }
from '@server/errors'; jest.mock('@server/lib/prisma', () => ({ prisma: {
inventoryItem: { create: jest.fn(), delete: jest.fn(), findUnique: jest.fn(),
findMany: jest.fn() }, user: { findUnique: jest.fn(), update:
jest.fn() }, $transaction: jest.fn(), }, })); const mockPrisma =
prisma as jest.Mocked<typeof prisma>; describe('InventoryService', () => {
let service: InventoryService; beforeEach(() => { service = new
InventoryService(prisma); jest.clearAllMocks(); }); //
// a) Add item to inventory

```

```

// ----- it('should add an item to
a user inventory and return the inventory entry', async () => {    const
mockEntry = { id: 'inv_1', userId: 'user_1', itemId: 'item_hat', quantity:
1 };    (mockPrisma.inventoryItem.create as
jest.Mock).mockResolvedValue(mockEntry);    const result = await
service.addItem('user_1', 'item_hat');    expect(result).toEqual(mockEntry);
expect(mockPrisma.inventoryItem.create).toHaveBeenCalledWith(    expect.ob
jectContaining({ data: expect.objectContaining({ userId: 'user_1', itemId:
'item_hat' }) })    );    });    //

// b) Remove item not in
inventory → ItemNotFoundError    //

it('should throw
ItemNotFoundError when removing an item the user does not own', async () => {
(mockPrisma.inventoryItem.findUnique as jest.Mock).mockResolvedValue(null);
await expect(service.removeItem('user_1',
'item_not_owned')).rejects.toThrow(ItemNotFoundError);    });    //

// c) HavenCoin transfer –
atomic update    // ----- it('should
update both user balances atomically on a valid HavenCoin transfer', async ()
=> {    // Simulate the $transaction executing both updates together
(mockPrisma.$transaction as jest.Mock).mockImplementation(async (fn) =>
fn(mockPrisma));    (mockPrisma.user.findUnique as
jest.Mock).mockResolvedValue({ id: 'user_1', havenCoins: 500 });
(mockPrisma.user.update as jest.Mock).mockResolvedValue({});    await
service.transferCoins({ fromUserId: 'user_1', toUserId: 'user_2', amount: 100
});    // Both the debit and credit must occur inside a single $transaction
call    expect(mockPrisma.$transaction).toHaveBeenCalledTimes(1);
expect(mockPrisma.user.update).toHaveBeenCalledTimes(2); // Debit sender,
credit receiver    });    //

// d) Insufficient balance →
InsufficientFundsError    // -----
it('should throw InsufficientFundsError when sender has insufficient
HavenCoins', async () => {    (mockPrisma.user.findUnique as
jest.Mock).mockResolvedValue({ id: 'user_1', havenCoins: 50 });    await
expect(    service.transferCoins({ fromUserId: 'user_1', toUserId:
'user_2', amount:
100 })    ).rejects.toThrow(InsufficientFundsError);    // Critically,
$transaction must NOT be called – no partial writes    expect(mockPrisma.
$transaction).not.toHaveBeenCalled();    });    //

// e) Negative coin transfer
→ rejected    // ----- it('should
reject a negative coin transfer amount (economy exploit prevention)', async
() => {    // A negative amount would effectively give coins to the "sender"
– a classic exploit    await
expect(    service.transferCoins({ fromUserId: 'user_1', toUserId:
'user_2', amount: -
500 })    ).rejects.toThrow('INVALID_AMOUNT');    });    //

// f) Transfer to self →
rejected    // ----- it('should
reject a coin transfer where sender and receiver are the same user', async ()
=> {    await expect(    service.transferCoins({ fromUserId: 'user_1',
toUserId: 'user_1', amount:
100 })    ).rejects.toThrow('SELF_TRANSFER');    });    //

// g) Weekly earnings cap –
earning over cap is capped, not rejected    //

it('should cap earnings at
the weekly maximum without rejecting the transaction', async () =>
{    const weeklyMax = 10000;    const alreadyEarned = 9900;    const
attemptedEarn = 500; // Would push total to 10400 – over cap
(mockPrisma.user.findUnique as jest.Mock).mockResolvedValue({    id:

```

```

'user_1',      weeklyEarnings: alreadyEarned,      weeklyEarningsCap:
weeklyMax,    });    (mockPrisma.user.update as
jest.Mock).mockResolvedValue({});    const result = await
service.earnCoins('user_1', attemptedEarn);    // Should earn only 100 (the
remaining cap), not 500    expect(result.coinsAwarded).toBe(100);
expect(result.cappedAt).toBe(weeklyMax);    });    //
// h) Daily login streak –
second claim same day rejected    //
// it('should reject a second
daily login streak claim on the same calendar day', async () => {    const
today = new Date();    (mockPrisma.user.findUnique as
jest.Mock).mockResolvedValue({    id: 'user_1',    lastLoginClaimDate:
today, // Already claimed today    });    await
expect(service.claimDailyLoginBonus('user_1')).rejects.toThrow('ALREADY_CLAIM
ED_TODAY');    });    });    8. Unit Tests – WorkshopService & CraftingQueue
MAC File: src/server/services/__tests__/workshopService.test.ts The
crafting system deducts materials synchronously when a craft is started, then
rewards the item asynchronously when it completes. These tests verify both
halves of that contract, including the guard against claiming a craft before
its timer has elapsed.    //
src/server/services/__tests__/workshopService.test.ts import
{ WorkshopService } from '../workshopService'; import { prisma } from
'@server/lib/prisma'; import { InsufficientMaterialsError, CraftNotReadyError
} from '@server/errors'; jest.mock('@server/lib/prisma', () => ({    prisma:
{    craftingQueue: { create: jest.fn(), findUnique: jest.fn(), delete:
jest.fn(), findMany: jest.fn() },    inventoryItem: { findMany: jest.fn(),
updateMany: jest.fn(), create: jest.fn() },    $transaction: jest.fn(),    },
}));    const mockPrisma = prisma as jest.Mocked<typeof prisma>;
describe('WorkshopService', () => {    let service: WorkshopService;
beforeEach(() => {    service = new WorkshopService(prisma);
jest.clearAllMocks();    // Default $transaction passes through to the
callback    (mockPrisma.$transaction as jest.Mock).mockImplementation(async
(fn) => fn(mockPrisma));    });    //
// a) Start craft with
sufficient materials → queue entry created    //
// it('should create a
CraftingQueue entry and deduct materials when crafting starts', async () => {
// Mock user having the required materials
(mockPrisma.inventoryItem.findMany as
jest.Mock).mockResolvedValue([    { itemId: 'wood', quantity: 5 },    {
itemId: 'stone', quantity: 3 },    ]);    const mockQueueEntry = { id:
'queue_1', userId: 'user_1', recipeId: 'bench', completesAt: new Date() };
(mockPrisma.craftingQueue.create as
jest.Mock).mockResolvedValue(mockQueueEntry);    const result = await
service.startCraft({ userId: 'user_1', recipeId: 'bench' });
expect(result.queueEntry).toEqual(mockQueueEntry);    // Materials must be
deducted immediately when crafting starts
expect(mockPrisma.inventoryItem.updateMany).toHaveBeenCalled();
expect(mockPrisma.craftingQueue.create).toHaveBeenCalledTimes(1);    });    //
// b) Start craft with
insufficient materials → error, no queue entry    //
// it('should throw
InsufficientMaterialsError and not create a queue entry when materials are
lacking', async () => {    // User only has 2 wood, but recipe needs 5
(mockPrisma.inventoryItem.findMany as jest.Mock).mockResolvedValue([ { itemId:
'wood', quantity: 2 } ]);    await expect(service.startCraft({ userId:
'user_1', recipeId: 'bench' })).rejects.toThrow(InsufficientMaterialsError);
// No queue entry created – no partial state written
expect(mockPrisma.craftingQueue.create).not.toHaveBeenCalled();    });    //
// c) Claim completed craft →

```

```

item added to inventory, queue entry removed //
it('should add crafted item
to inventory and remove queue entry when craft is complete', async () => {
// completesAt is in the past → craft is done    const pastDate = new
Date(Date.now() - 1000);    (mockPrisma.craftingQueue.findUnique as
jest.Mock).mockResolvedValue({    id: 'queue_1', userId: 'user_1',
recipeId: 'bench', completesAt: pastDate, outputItemId: 'item_bench',    });
await service.claimCraft({ userId: 'user_1', queueId: 'queue_1' });
expect(mockPrisma.inventoryItem.create).toHaveBeenCalledWith(    expect.ob
jectContaining({ data: expect.objectContaining({ itemId: 'item_bench',
userId: 'user_1' }) })    );
expect(mockPrisma.craftingQueue.delete).toHaveBeenCalledWith(    expect.ob
jectContaining({ where: { id: 'queue_1' } })    );    });    //
// d) Claim craft not yet
done → CraftNotReadyError    //
it('should throw CraftNotReadyError when attempting to claim before
completion time', async () => {    // completesAt is in the future → craft
is still in progress    const futureDate = new Date(Date.now() + 60000);
(mockPrisma.craftingQueue.findUnique as
jest.Mock).mockResolvedValue({    id: 'queue_1', userId: 'user_1',
recipeId: 'bench', completesAt: futureDate,    });    await
expect(service.claimCraft({ userId: 'user_1', queueId:
'queue_1' })).rejects.toThrow(CraftNotReadyError);    // No inventory write
should occur
expect(mockPrisma.inventoryItem.create).not.toHaveBeenCalled();    });    //
// e) Multiple simultaneous
crafts → independent queue slots    //
it('should create independent
queue entries for multiple simultaneous crafts', async () =>
{    (mockPrisma.inventoryItem.findMany as jest.Mock).mockResolvedValue([
{ itemId: 'wood', quantity: 20 }, // Plenty for multiple crafts    ]);
(mockPrisma.craftingQueue.create as jest.Mock).mockResolvedValueOnce({
id: 'queue_1', recipeId: 'plank' })    .mockResolvedValueOnce({ id:
'queue_2', recipeId: 'plank' });    const [r1, r2] = await Promise.all([
service.startCraft({ userId: 'user_1', recipeId: 'plank' }),
service.startCraft({ userId: 'user_1', recipeId: 'plank' }),    ]);    //
Each craft gets a distinct queue entry ID
expect(r1.queueEntry.id).not.toBe(r2.queueEntry.id);
expect(mockPrisma.craftingQueue.create).toHaveBeenCalledTimes(2);    });    });
9. Unit Tests – TradeManager (P2P Trade) MAC File:
src/server/services/__tests__/tradeManager.test.ts The peer-to-peer trade
system is the highest-risk economy feature. The race condition test (case f)
is critical: if two simultaneous confirmations both pass the balance check
before either writes, one player can receive items without losing theirs.
Only a Prisma $transaction with optimistic locking prevents this.    //
src/server/services/__tests__/tradeManager.test.ts import { TradeManager }
from '../tradeManager'; import { prisma } from '@server/lib/prisma';
jest.mock('@server/lib/prisma', () => ({    prisma: {    trade:
{
create: jest.fn(),    findUnique: jest.fn(),    update:
jest.fn(),    delete: jest.fn(),    },    $transaction:
jest.fn(),    },    }));    const mockPrisma = prisma as jest.Mocked<typeof
prisma>; describe('TradeManager', () => {    let manager: TradeManager;    let
mockEmit: jest.Mock;    beforeEach(() => {    mockEmit = jest.fn();
manager = new TradeManager(prisma, mockEmit); // Inject mock event emitter
jest.clearAllMocks();    jest.useFakeTimers(); // Control timeout behavior
in tests    });    afterEach(() => {    jest.useRealTimers();    });    //
// a) Initiate trade – same
room → TRADE_INITIATED sent to both    //
it('should emit
TRADE_INITIATED to both users when they are in the same room', async () => {

```

```

(mockPrisma.trade.create as jest.Mock).mockResolvedValue({ id: 'trade_1',
state: 'PENDING' });      await manager.initiateTrade({ initiatorId:
'user_1', targetId: 'user_2', roomId: 'town_square' });
expect(mockEmit).toHaveBeenCalledWith('user_1', 'TRADE_INITIATED',
expect.objectContaining({ tradeId: 'trade_1' }));
expect(mockEmit).toHaveBeenCalledWith('user_2', 'TRADE_INITIATED',
expect.objectContaining({ tradeId: 'trade_1' }));  });  //
// b) Initiate trade –
different rooms → rejected  // _____
it('should reject trade initiation when users are in different rooms', async
() => {      // TradeManager checks room presence before creating a trade
record      await expect(      manager.initiateTrade({ initiatorId: 'user_1',
targetId: 'user_2', roomId: 'town_square',      targetRoomId:
'fishing_dock' }) // Different
room      ).rejects.toThrow('USERS_NOT_IN_SAME_ROOM');
expect(mockPrisma.trade.create).not.toHaveBeenCalled();  });  //
// c) Both users lock → state
LOCKED  // _____      it('should
transition trade state to LOCKED when both users lock their offers', async ()
=> {      (mockPrisma.trade.findUnique as jest.Mock).mockResolvedValue({ id:
'trade_1', state: 'PENDING', lockedBy: ['user_1'] });
(mockPrisma.trade.update as jest.Mock).mockResolvedValue({ id: 'trade_1',
state: 'LOCKED' });      await manager.lockOffer({ tradeId: 'trade_1',
userId: 'user_2' });
expect(mockPrisma.trade.update).toHaveBeenCalledWith(      expect.objectCont
aining({ data: expect.objectContaining({ state: 'LOCKED' }) })      );  });
// _____      // d) One user unlocks
after lock → state reverts to PENDING  //
_____      it('should revert trade state
to PENDING when one user unlocks after both have locked', async () =>
{      (mockPrisma.trade.findUnique as jest.Mock).mockResolvedValue({ id:
'trade_1', state: 'LOCKED', lockedBy: ['user_1', 'user_2'] });
(mockPrisma.trade.update as jest.Mock).mockResolvedValue({ id: 'trade_1',
state: 'PENDING' });      await manager.unlockOffer({ tradeId: 'trade_1',
userId: 'user_1' });
expect(mockPrisma.trade.update).toHaveBeenCalledWith(      expect.objectCont
aining({ data: expect.objectContaining({ state: 'PENDING' }) })      );  });
// _____      // e) Both confirm after
lock → atomic swap executes  //
_____      it('should execute an atomic
item swap via $transaction when both users confirm', async () =>
{      (mockPrisma.trade.findUnique as
jest.Mock).mockResolvedValue({      id: 'trade_1', state: 'LOCKED',
confirmedBy: ['user_1'], initiatorItems: ['item_a'], targetItems: ['item_b'],
});      (mockPrisma.$transaction as jest.Mock).mockResolvedValue(true);
await manager.confirmTrade({ tradeId: 'trade_1', userId: 'user_2' });      //
The swap must be wrapped in a single $transaction – no partial transfers
expect(mockPrisma.$transaction).toHaveBeenCalledTimes(1);  });  //
// f) Race condition:
simultaneous confirms → exactly one swap  //
_____      it('should process only one
swap even when both users confirm simultaneously', async () => {      //
Simulate both users hitting confirmTrade at the exact same millisecond
(mockPrisma.trade.findUnique as jest.Mock).mockResolvedValue({ id: 'trade_1',
state: 'LOCKED', confirmedBy: [] });      (mockPrisma.$transaction as
jest.Mock).mockResolvedValueOnce(true).mockRejectedValueOnce(new
Error('CONFLICT'));      const [r1, r2] = await
Promise.allSettled([      manager.confirmTrade({ tradeId: 'trade_1', userId:
'user_1' }),
manager.confirmTrade({ tradeId: 'trade_1', userId:
'user_1' }),      ]);      // Exactly one $transaction call must succeed; the

```

```

second must be rejected (idempotency)    const successes = [r1, r2].filter(r
=> r.status === 'fulfilled');    expect(successes.length).toBe(1);
expect(mockPrisma.$transaction).toHaveBeenCalledTimes(1);    });    //
// g) Cancel trade at any
state → no items transferred    //
    it('should cancel a trade and
transfer no items regardless of current state', async () =>
{    (mockPrisma.trade.findUnique as jest.Mock).mockResolvedValue({ id:
'trade_1', state: 'LOCKED' });    (mockPrisma.trade.delete as
jest.Mock).mockResolvedValue({});    await manager.cancelTrade({ tradeId:
'trade_1', cancelledBy: 'user_1' });
expect(mockPrisma.trade.delete).toHaveBeenCalledWith({ where: { id: 'trade_1'
} });    expect(mockPrisma.$transaction).not.toHaveBeenCalled(); // No swap
happened    });    //    // h)
Trade timeout after 60 seconds → auto-cancelled    //
    it('should auto-cancel a
trade that has been inactive for 60 seconds', async () =>
{    (mockPrisma.trade.create as jest.Mock).mockResolvedValue({ id:
'trade_1', state: 'PENDING' });    (mockPrisma.trade.delete as
jest.Mock).mockResolvedValue({});    await
manager.initiateTrade({ initiatorId: 'user_1', targetId: 'user_2', roomId:
'town_square' });    // Advance Jest fake timers by 60 seconds
jest.advanceTimersByTime(60000);    // The timeout callback should have
fired and deleted the trade
expect(mockPrisma.trade.delete).toHaveBeenCalledWith({ where: { id: 'trade_1'
} });    expect(mockEmit).toHaveBeenCalledWith('user_1', 'TRADE_CANCELLED',
expect.objectContaining({ reason: 'TIMEOUT' }));
expect(mockEmit).toHaveBeenCalledWith('user_2', 'TRADE_CANCELLED',
expect.objectContaining({ reason: 'TIMEOUT' }));    });    });    10.
Integration Tests – Full Auth Flow MAC    File:
__tests__/integration/auth.integration.test.ts    Integration tests use
Supertest to fire real HTTP requests against the Express application. Unlike
unit tests, these hit the actual havenworld_test PostgreSQL database, proving
that routes, middleware, services, and the database cooperate correctly as a
system.    // __tests__/integration/auth.integration.test.ts import request
from 'supertest'; import { app } from '@server/app'; // The Express app
instance (no server.listen called) import { prisma } from
'@server/lib/prisma'; import { createTestUser, createTestInviteCode } from
'@tests/helpers/factories'; import { truncateAllTables } from
'@tests/helpers/dbHelpers'; describe('Auth Integration – Full HTTP Flow', ()
=> {    beforeAll(async () => {    // Start with a clean slate for this test
suite    await truncateAllTables();    });    afterAll(async () =>
{    await prisma.$disconnect();    });    //
// a) Register with valid
invite code → 201, user in DB    //
    it('should register a new
user and return 201 when given a valid unused invite code', async () => {
const invite = await createTestInviteCode({ code: 'VALID_CODE_001', used:
false });    const res = await request(app)
.post('/api/auth/register')    .send({ email: 'alice@test.com', username:
'alice', password: 'SecurePass1!', inviteCode: 'VALID_CODE_001' });
expect(res.status).toBe(201);    expect(res.body).toHaveProperty('message');
// Verify user actually exists in the test database    const dbUser = await
prisma.user.findUnique({ where: { email: 'alice@test.com' } });
expect(dbUser).not.toBeNull();    expect(dbUser!.emailVerified).toBe(false);
// Pending email verification    });    //
// b) Register with used
invite code → 400    //
it('should return 400 when registering with an already-used invite code',
async () => {    await createTestInviteCode({ code: 'USED_CODE_001', used:

```

```

true });    const res = await request(app)
.post('/api/auth/register')      .send({ email: 'bob@test.com', username:
'bob', password: 'SecurePass1!', inviteCode: 'USED_CODE_001' });
expect(res.status).toBe(400);    expect(res.body.error).toMatch(/invite/i);
});    // ----- // c) Duplicate
email → 409    // -----    it('should
return 409 when registering with an email that already exists', async () => {
await createTestUser({ email: 'existing@test.com' });    await
createTestInviteCode({ code: 'FRESH_CODE_001', used: false });    const res
= await request(app)      .post('/api/auth/register')      .send({ email:
'existing@test.com', username: 'newuser', password: 'SecurePass1!',
inviteCode: 'FRESH_CODE_001' });
expect(res.status).toBe(409);    });    //

// d) Email verification with
valid token → 200    // -----
it('should verify email and return 200 when given a valid verification
token', async () => {    const user = await createTestUser({ email:
'verify@test.com', emailVerified: false, verificationToken: 'tok_abc' });
const res = await request(app).get('/api/auth/verify?token=tok_abc');
expect(res.status).toBe(200);    const updated = await
prisma.user.findUnique({ where: { id: user.id } });
expect(updated!.emailVerified).toBe(true);    });    //

// e) Login with valid
credentials → 200, tokens set    // -----
it('should return 200 with
accessToken and set httpOnly refresh cookie on valid login', async () => {
await createTestUser({ email: 'login@test.com', password: 'SecurePass1!',
emailVerified: true });    const res = await request(app)
.post('/api/auth/login')      .send({ email: 'login@test.com', password:
'SecurePass1!' });    expect(res.status).toBe(200);
expect(res.body).toHaveProperty('accessToken');    // Refresh token must be
in httpOnly cookie, never in response body    const cookies =
res.headers['set-cookie'] as string[];    expect(cookies.some((c: string) =>
c.includes('refreshToken') &&
c.includes('HttpOnly'))).toBe(true);    });    //

// f) Login with wrong
password → 401    // -----
it('should return 401 when login is attempted with an incorrect password',
async () => {    await createTestUser({ email: 'wrongpass@test.com',
emailVerified: true });    const res = await request(app)
.post('/api/auth/login')      .send({ email: 'wrongpass@test.com', password:
'WrongPassword!' });    expect(res.status).toBe(401);    });    //

// g) Login with unverified
account → 403    // -----    it('should
return 403 when a user with an unverified email attempts to login', async ()
=> {    await createTestUser({ email: 'unverified@test.com', emailVerified:
false });    const res = await request(app)      .post('/api/auth/login')
.send({ email: 'unverified@test.com', password: 'SecurePass1!' });
expect(res.status).toBe(403);    expect(res.body.error).toMatch(/verif/i);
});    // ----- // h) Refresh with
valid cookie → 200, new accessToken    // -----
it('should issue a new
accessToken when a valid refresh cookie is provided', async () => {    //
First, perform a real login to get a real refresh cookie    await
createTestUser({ email: 'refresh@test.com', emailVerified: true });    const
loginRes = await request(app)      .post('/api/auth/login')      .send({ email: 'refresh@test.com', password:
'SecurePass1!' });    const cookies = loginRes.headers['set-cookie'];
const res = await request(app)      .post('/api/auth/refresh')      .set('Cookie', cookies);

```

```

expect(res.status).toBe(200); // New accessToken must be different from
the login token (rotated)
expect(res.body.accessToken).not.toBe(loginRes.body.accessToken); }); //
// i) Refresh with expired
refresh token → 401 // -----
it('should return 401 when the refresh token cookie has expired', async () =>
{
  const res = await request(app)
    .post('/api/auth/refresh')
    .set('Cookie',
      'refreshToken=expired.token.here; HttpOnly');
  expect(res.status).toBe(401); }); //
// j) Logout → refresh cookie
cleared // ----- it('should clear
the refresh cookie on logout', async () => {
  await
  createTestUser({ email: 'logout@test.com', emailVerified: true });
  const
  loginRes = await request(app)
    .post('/api/auth/login')
    .send({ email: 'logout@test.com', password:
      'SecurePass1!' });
  const res = await request(app)
    .post('/api/auth/logout')
    .set('Cookie', loginRes.headers['set-
      cookie']);
  // Cookie must be cleared (Max-Age=0 or Expires in the past)
  const setCookieHeader = res.headers['set-cookie'] as string[];
  expect(setCookieHeader.some((c: string) => c.includes('refreshToken') &&
    c.includes('Max-Age=0'))).toBe(true); }); // 11. Integration Tests –
Socket.io Connection & Room Flow MAC File:
__tests__/integration/socketio.integration.test.ts
Socket.io integration
tests create a real HTTP server in-process, attach the Socket.io instance to
it, then connect socket.io-client instances. This is the most realistic test
short of a real browser, with zero transport-level mocking. //
__tests__/integration/socketio.integration.test.ts
import { createServer }
from 'http'; import { Server as SocketIOServer } from 'socket.io'; import
{ io as ClientIO, Socket } from 'socket.io-client'; import { createApp } from
'@server/app'; import { attachSocketIO } from '@server/socket'; import
{ generateTestToken } from '@tests/helpers/jwtHelpers'; import
{ createTestUser, createTestRoom } from '@tests/helpers/factories'; import
{ createAuthenticatedSocket, waitForEvent, closeAllSockets } from
'@tests/helpers/socketHelpers'; describe('Socket.io Integration – Connection
& Room Flow', () => {
  let httpServer: ReturnType<typeof createServer>;
  let ioServer: SocketIOServer; let serverUrl: string; let sockets:
  Socket[] = [];
  beforeAll(async () => {
    const app = createApp();
    httpServer = createServer(app);
    ioServer = attachSocketIO(httpServer);
    await new Promise<void>(resolve => httpServer.listen(0, resolve));
    const
    addr = httpServer.address() as { port: number };
    serverUrl =
    `http://localhost:${addr.port}`;
  });
  afterEach(async () => {
    //
    Close all sockets created during a test to prevent handle leaks
    await
    closeAllSockets(sockets);
    sockets = [];
  });
  afterAll(async () => {
    await new Promise(resolve => httpServer.close(resolve));
  }); //
// a) Connect without auth →
connect_error // ----- it('should
emit connect_error AUTH_REQUIRED when connecting without an auth token',
async () => {
  const socket = ClientIO(serverUrl, { auth: {} }); // No
  token
  sockets.push(socket);
  const error = await waitForEvent(socket,
    'connect_error');
  expect(error.message).toMatch(/AUTH_REQUIRED/);
}); //
// b) Connect with valid
token → WELCOME event // -----
it('should connect successfully and receive WELCOME event with valid JWT',
async () => {
  const user = await createTestUser();
  const token =
  generateTestToken(user.id);
  const socket = ClientIO(serverUrl, { auth: {
    token } });
  sockets.push(socket);
  const welcome = await
  waitForEvent(socket, 'WELCOME');
  expect(welcome).toHaveProperty('userId',
    user.id);
  expect(socket.id).toBeDefined();
}); //
// c) JOIN_ROOM valid room →

```

```

ROOM_STATE received // _____
it('should receive ROOM_STATE with a users array when joining a valid room',
  async () => {
    const user = await createTestUser();
    const room = await createTestRoom(user.id);
    const socket = await createAuthenticatedSocket(serverUrl, user.id, sockets);
    socket.emit('JOIN_ROOM', { roomId: room.id });
    const roomState = await waitForEvent(socket, 'ROOM_STATE');
    expect(roomState).toHaveProperty('roomId', room.id);
    expect(Array.isArray(roomState.users)).toBe(true);
  }); //
// d) PLAYER_MOVE → other
clients receive PLAYER_MOVED //
// _____
it('should broadcast PLAYER_MOVED to other room clients when PLAYER_MOVE is emitted', async () => {
  const [user1, user2] = await Promise.all([createTestUser(), createTestUser()]);
  const room = await createTestRoom(user1.id);
  const [socket1, socket2] = await Promise.all([
    createAuthenticatedSocket(serverUrl, user1.id, sockets),
    createAuthenticatedSocket(serverUrl, user2.id, sockets),
  ]); // Both join the same room
  socket1.emit('JOIN_ROOM', { roomId: room.id });
  socket2.emit('JOIN_ROOM', { roomId: room.id });
  await new Promise(r => setTimeout(r, 200)); // Allow joins to settle
  socket1.emit('PLAYER_MOVE', { roomId: room.id, x: 150, y: 200 });
  const moved = await waitForEvent(socket2, 'PLAYER_MOVED');
  expect(moved).toMatchObject({ userId: user1.id, x: 150, y: 200 });
}); // _____ // e)
SEND_CHAT valid message → room receives CHAT_MESSAGE //
// _____
it('should broadcast CHAT_MESSAGE with username and timestamp to the room', async () => {
  const [user1, user2] = await Promise.all([createTestUser(), createTestUser()]);
  const room = await createTestRoom(user1.id);
  const [socket1, socket2] = await Promise.all([
    createAuthenticatedSocket(serverUrl, user1.id, sockets),
    createAuthenticatedSocket(serverUrl, user2.id, sockets),
  ]);
  socket1.emit('JOIN_ROOM', { roomId: room.id });
  socket2.emit('JOIN_ROOM', { roomId: room.id });
  await new Promise(r => setTimeout(r, 200));
  socket1.emit('SEND_CHAT', { roomId: room.id, message: 'Hello everyone!' });
  const chat = await waitForEvent(socket2, 'CHAT_MESSAGE');
  expect(chat).toMatchObject({ username: user1.username, message: 'Hello everyone!' });
  expect(chat).toHaveProperty('timestamp');
}); // _____ // f) SEND_CHAT with
profanity → sender gets error, room gets nothing //
// _____
it('should send error to sender and send nothing to room when chat contains profanity', async () => {
  const [user1, user2] = await Promise.all([createTestUser(), createTestUser()]);
  const room = await createTestRoom(user1.id);
  const [socket1, socket2] = await Promise.all([
    createAuthenticatedSocket(serverUrl, user1.id, sockets),
    createAuthenticatedSocket(serverUrl, user2.id, sockets),
  ]);
  socket1.emit('JOIN_ROOM', { roomId: room.id });
  socket2.emit('JOIN_ROOM', { roomId: room.id });
  await new Promise(r => setTimeout(r, 200));
  let socket2Received = false;
  socket2.on('CHAT_MESSAGE', () => {
    socket2Received = true;
  });
  socket1.emit('SEND_CHAT', { roomId: room.id, message: '[profanity] word here' });
  const error = await waitForEvent(socket1, 'CHAT_ERROR');
  expect(error.reason).toBe('PROFANITY_DETECTED');
  await new Promise(r => setTimeout(r, 300));
  expect(socket2Received).toBe(false); // Room received nothing
}); // _____ // g) Disconnect → other
users receive USER_LEFT // _____
it('should broadcast USER_LEFT to remaining room users when a socket

```

```

disconnects', async () => {      const [user1, user2] = await
Promise.all([createTestUser(), createTestUser()]);      const room = await
createTestRoom(user1.id);      const [socket1, socket2] = await Promise.all([
createAuthenticatedSocket(serverUrl, user1.id, sockets),
createAuthenticatedSocket(serverUrl, user2.id, sockets),      ]));
socket1.emit('JOIN_ROOM', { roomId: room.id });      socket2.emit('JOIN_ROOM',
{ roomId: room.id });      await new Promise(r => setTimeout(r, 200));      //
Disconnect user1 – user2 must be notified      const leftPromise =
waitForEvent(socket2, 'USER_LEFT');      socket1.disconnect();      const left
= await leftPromise;      expect(left.userId).toBe(user1.id);      });      //
// h) Two clients in same
room → each receives other's PLAYER_MOVED      //
      it('should deliver
PLAYER_MOVED events bidirectionally between two clients in the same room',
async () => {      const [user1, user2] = await Promise.all([createTestUser(),
createTestUser()]);      const room = await createTestRoom(user1.id);
const [socket1, socket2] = await
Promise.all([      createAuthenticatedSocket(serverUrl, user1.id, sockets),
createAuthenticatedSocket(serverUrl, user2.id, sockets),      ]));
socket1.emit('JOIN_ROOM', { roomId: room.id });      socket2.emit('JOIN_ROOM',
{ roomId: room.id });      await new Promise(r => setTimeout(r, 200));      //
Test bidirectional delivery      socket1.emit('PLAYER_MOVE', { roomId:
room.id, x: 10, y: 10 });      const move1 = await waitForEvent(socket2,
'PLAYER_MOVED');      expect(move1.userId).toBe(user1.id);
socket2.emit('PLAYER_MOVE', { roomId: room.id, x: 20, y: 20 });      const
move2 = await waitForEvent(socket1, 'PLAYER_MOVED');
expect(move2.userId).toBe(user2.id);      });      });      12. Integration Tests –
Fishing Mini-Game Server Validation MAC File:
__tests__/integration/fishing.integration.test.ts The fishing mini-game is
an economy source. Every test here protects against reward duplication and
session confusion. The duplicate reward test is especially important: a
client bug or network retry could fire CAST_COMPLETE twice, and only server-
side session tracking prevents double rewards.      //
__tests__/integration/fishing.integration.test.ts import { createServer }
from 'http'; import { Server as SocketIOServer } from 'socket.io'; import
{ createApp } from '@server/app'; import { attachSocketIO } from
'@server/socket'; import { createTestUser, createTestRoom } from
'@tests/helpers/factories'; import { createAuthenticatedSocket, waitForEvent,
closeAllSockets } from '@tests/helpers/socketHelpers'; import { prisma } from
'@server/lib/prisma'; import { Socket } from 'socket.io-client';
describe('Fishing Mini-Game Integration', () => {      let httpServer:
ReturnType<typeof createServer>;      let serverUrl: string;      let sockets:
Socket[] = [];      beforeAll(async () => {      const app = createApp();
httpServer = createServer(app);      attachSocketIO(httpServer);      await new
Promise<void>(resolve => httpServer.listen(0, resolve));      const addr =
httpServer.address() as { port: number };      serverUrl = `http://localhost:${
addr.port}`;      });      afterEach(async () => {      await
closeAllSockets(sockets);      sockets = [];      });      afterAll(async () => {
await new Promise(resolve => httpServer.close(resolve));      await prisma.
$disconnect();      });      // _____      //
a) START_FISHING → server sends FISH_ON within timeout      //
      it('should receive FISH_ON
event within 10 seconds of emitting START_FISHING', async () => {      const
user = await createTestUser();      const room = await createTestRoom(user.id,
{ type: 'FISHING_DOCK' });      const socket = await
createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });      await new Promise(r =>
setTimeout(r, 100));      socket.emit('START_FISHING', { roomId: room.id });
// Server must confirm a fish is on the line before the client times out
const fishOn = await waitForEvent(socket, 'FISH_ON', 10000);

```

```

expect(fishOn).toHaveProperty('tensionTarget'); // Sweet spot data for client
UI  }); // _____ // b)
CAST_COMPLETE with valid tension → FISH_CAUGHT + reward //
_____ it('should emit FISH_CAUGHT
with coin reward when tension is within the sweet spot', async () =>
{   const user = await createTestUser();   const room = await
createTestRoom(user.id, { type: 'FISHING_DOCK' });   const socket = await
createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });
socket.emit('START_FISHING', { roomId: room.id });   const fishOn = await
waitForEvent(socket, 'FISH_ON', 10000);   // Send tension exactly at the
middle of the sweet spot – should always succeed
socket.emit('CAST_COMPLETE', { roomId: room.id, tension: fishOn.tensionTarget
});   const caught = await waitForEvent(socket, 'FISH_CAUGHT', 5000);
expect(caught).toHaveProperty('reward');
expect(caught.reward).toBeGreaterThan(0);  }); //
_____ // c) CAST_COMPLETE with high
tension → FISH_ESCAPED, no reward //
_____ it('should emit FISH_ESCAPED
with no reward when tension is too high', async () => {   const user =
await createTestUser();   const room = await createTestRoom(user.id,
{ type: 'FISHING_DOCK' });   const socket = await
createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });
socket.emit('START_FISHING', { roomId: room.id });   await
waitForEvent(socket, 'FISH_ON', 10000);   // Max tension (1.0) is always
outside the sweet spot   socket.emit('CAST_COMPLETE', { roomId: room.id,
tension: 1.0 });   const escaped = await waitForEvent(socket,
'FISH_ESCAPED', 5000);
expect(escaped).not.toHaveProperty('reward');  }); //
_____ // d) CAST_COMPLETE without
prior START_FISHING → rejected //
_____ it('should reject
CAST_COMPLETE when no active fishing session exists', async () => {   const
user = await createTestUser();   const room = await createTestRoom(user.id,
{ type: 'FISHING_DOCK' });   const socket = await
createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });   // Send cast without
starting – no active session on the server   socket.emit('CAST_COMPLETE', {
roomId: room.id, tension: 0.5 });   const error = await
waitForEvent(socket, 'FISHING_ERROR', 3000);
expect(error.reason).toBe('NO_ACTIVE_SESSION');  }); //
_____ // e) Duplicate CAST_COMPLETE
from same user → only one reward //
_____ it('should award coins only
once even when CAST_COMPLETE is sent twice rapidly', async () => {   const
user = await createTestUser();   const room = await createTestRoom(user.id,
{ type: 'FISHING_DOCK' });   const socket = await
createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });
socket.emit('START_FISHING', { roomId: room.id });   const fishOn = await
waitForEvent(socket, 'FISH_ON', 10000);   const beforeCoins = (await
prisma.user.findUnique({ where: { id: user.id } })).havenCoins;   // Fire
CAST_COMPLETE twice in the same tick (simulating client retry)
socket.emit('CAST_COMPLETE', { roomId: room.id, tension: fishOn.tensionTarget
});   socket.emit('CAST_COMPLETE', { roomId: room.id, tension:
fishOn.tensionTarget });   await new Promise(r => setTimeout(r, 1000));
const afterCoins = (await prisma.user.findUnique({ where: { id:
user.id } })).havenCoins;   // Coins must increase by exactly the single-
catch reward, not double   const coinsAwarded = afterCoins - beforeCoins;

```

```

expect(coinsAwarded).toBeLessThanOrEqual(fishOn.maxReward ||
100); }); // _____ // f) Fish
reward respects WeeklyEarningsCap //
_____ it('should not award coins
beyond the remaining WeeklyEarningsCap balance', async () => { // Create
user who is near their weekly cap const user = await
createTestUser({ weeklyEarnings: 9990, weeklyEarningsCap: 10000 }); const
room = await createTestRoom(user.id, { type: 'FISHING_DOCK' }); const
socket = await createAuthenticatedSocket(serverUrl, user.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id });
socket.emit('START_FISHING', { roomId: room.id }); const fishOn = await
waitForEvent(socket, 'FISH_ON', 10000); socket.emit('CAST_COMPLETE',
{ roomId: room.id, tension: fishOn.tensionTarget }); const caught =
await waitForEvent(socket, 'FISH_CAUGHT', 5000); // Max possible award is
10 (cap remaining), not the full catch reward
expect(caught.reward).toBeLessThanOrEqual(10); }); }); 13. Integration
Tests – Loft Privacy & Doorbell MAC File:
__tests__/integration/loft.integration.test.ts //
__tests__/integration/loft.integration.test.ts import { createServer } from
'http'; import { createApp } from '@server/app'; import { attachSocketIO }
from '@server/socket'; import { createTestUser, createTestRoom } from
'@tests/helpers/factories'; import { createAuthenticatedSocket, waitForEvent,
closeAllSockets } from '@tests/helpers/socketHelpers'; import { prisma } from
'@server/lib/prisma'; import { Socket } from 'socket.io-client';
describe('Loft Privacy & Doorbell Integration', () => { let httpServer:
ReturnType<typeof createServer>; let serverUrl: string; let sockets:
Socket[] = []; beforeAll(async () => { const app = createApp();
httpServer = createServer(app); attachSocketIO(httpServer); await new
Promise<void>(resolve => httpServer.listen(0, resolve)); const addr =
httpServer.address() as { port: number }; serverUrl = `http://localhost:$
{addr.port}`; }); afterEach(async () => { await
closeAllSockets(sockets); sockets = []; }); afterAll(async () => {
await new Promise(resolve => httpServer.close(resolve)); await prisma.
$disconnect(); }); // _____ //
a) Public loft → any user can JOIN_ROOM //
_____ it('should allow any user to
join a public loft', async () => { const owner = await createTestUser();
const stranger = await createTestUser(); const room = await
createTestRoom(owner.id, { privacy: 'PUBLIC' }); const socket = await
createAuthenticatedSocket(serverUrl, stranger.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id }); const state = await
waitForEvent(socket, 'ROOM_STATE', 3000);
expect(state.roomId).toBe(room.id); }); //
_____ // b) Friends-only loft →
stranger gets ROOM_ACCESS_DENIED //
_____ it('should deny a non-friend
entry to a friends-only loft', async () => { const owner = await
createTestUser(); const stranger = await createTestUser(); // No
friendship record created const room = await createTestRoom(owner.id,
{ privacy: 'FRIENDS_ONLY' }); const socket = await
createAuthenticatedSocket(serverUrl, stranger.id, sockets);
socket.emit('JOIN_ROOM', { roomId: room.id }); const denied = await
waitForEvent(socket, 'ROOM_ACCESS_DENIED', 3000);
expect(denied.reason).toBe('NOT_FRIEND'); }); //
_____ // c) Password-protected loft
– correct/wrong password // _____
it('should allow entry with correct password and deny with wrong password',
async () => { const owner = await createTestUser(); const user =
await createTestUser(); const room = await createTestRoom(owner.id,
{ privacy: 'PASSWORD', password: 'secret123' }); const socketRight =

```

```

await createAuthenticatedSocket(serverUrl, user.id, sockets);
socketRight.emit('JOIN_ROOM', { roomId: room.id, password: 'secret123' });
const state = await waitForEvent(socketRight, 'ROOM_STATE', 3000);
expect(state.roomId).toBe(room.id);      const user2 = await
createTestUser();      const socketWrong = await
createAuthenticatedSocket(serverUrl, user2.id, sockets);
socketWrong.emit('JOIN_ROOM', { roomId: room.id, password: 'wrongpass' });
const denied = await waitForEvent(socketWrong, 'ROOM_ACCESS_DENIED', 3000);
expect(denied.reason).toBe('WRONG_PASSWORD');  });  //
// d) Locked loft →
RING_DOORBELL queues knock, owner gets DOORBELL_KNOCK  //
it('should queue a doorbell
knock and notify the owner when a user rings a locked loft', async () => {
const owner = await createTestUser();      const visitor = await
createTestUser();      const room = await createTestRoom(owner.id, { privacy:
'LOCKED' });      const ownerSocket = await
createAuthenticatedSocket(serverUrl, owner.id, sockets);      const
visitorSocket = await createAuthenticatedSocket(serverUrl, visitor.id,
sockets);      ownerSocket.emit('JOIN_ROOM', { roomId: room.id }); // Owner
is inside      await new Promise(r => setTimeout(r, 100));
visitorSocket.emit('RING_DOORBELL', { roomId: room.id });      const knock =
await waitForEvent(ownerSocket, 'DOORBELL_KNOCK', 3000);
expect(knock.visitorId).toBe(visitor.id);  });  //
// e) Owner approves knock →
visitor admitted  //
it('should admit the visitor when the owner approves a doorbell knock', async
() => {      const owner = await createTestUser();      const visitor = await
createTestUser();      const room = await createTestRoom(owner.id, { privacy:
'LOCKED' });      const ownerSocket = await
createAuthenticatedSocket(serverUrl, owner.id, sockets);      const
visitorSocket = await createAuthenticatedSocket(serverUrl, visitor.id,
sockets);      ownerSocket.emit('JOIN_ROOM', { roomId: room.id });      await
new Promise(r => setTimeout(r, 100));
visitorSocket.emit('RING_DOORBELL', { roomId: room.id });      const knock =
await waitForEvent(ownerSocket, 'DOORBELL_KNOCK', 3000);
ownerSocket.emit('DOORBELL_DECISION', { roomId: room.id, visitorId:
visitor.id, decision: 'allow' });      // Visitor should now receive
ROOM_STATE (they were admitted)      const state = await
waitForEvent(visitorSocket, 'ROOM_STATE', 3000);
expect(state.roomId).toBe(room.id);  });  //
// f) Owner rejects knock →
visitor gets ENTRY_REJECTED  //
it('should send
ENTRY_REJECTED to the visitor when the owner rejects the doorbell knock',
async () => {      const owner = await createTestUser();      const visitor =
await createTestUser();      const room = await createTestRoom(owner.id,
{ privacy: 'LOCKED' });      const ownerSocket = await
createAuthenticatedSocket(serverUrl, owner.id, sockets);      const
visitorSocket = await createAuthenticatedSocket(serverUrl, visitor.id,
sockets);      ownerSocket.emit('JOIN_ROOM', { roomId: room.id });      await
new Promise(r => setTimeout(r, 100));      visitorSocket.emit('RING_DOORBELL',
{ roomId: room.id });      await waitForEvent(ownerSocket, 'DOORBELL_KNOCK',
3000);      ownerSocket.emit('DOORBELL_DECISION', { roomId: room.id,
visitorId: visitor.id, decision: 'deny' });      const rejected = await
waitForEvent(visitorSocket, 'ENTRY_REJECTED', 3000);
expect(rejected.roomId).toBe(room.id);  });  //
// g) Co-decorator
permissions – PLACE_FURNITURE  //
it('should allow furniture
placement for co-decorators and deny non-granted users', async () =>

```

```

{
  const owner = await createTestUser();
  const coDecorator = await createTestUser();
  const stranger = await createTestUser();
  const room = await createTestRoom(owner.id, { privacy: 'PUBLIC' });
  // Grant co-decorator rights in DB
  await prisma.roomPermission.create({ data: {
    roomId: room.id,
    userId: coDecorator.id,
    role: 'CO_DECORATOR'
  }});
  const decoSocket = await createAuthenticatedSocket(serverUrl, coDecorator.id, sockets);
  const strangerSocket = await createAuthenticatedSocket(serverUrl, stranger.id, sockets);
  decoSocket.emit('JOIN_ROOM', { roomId: room.id });
  strangerSocket.emit('JOIN_ROOM', { roomId: room.id });
  await new Promise(r => setTimeout(r, 200));
  decoSocket.emit('PLACE_FURNITURE', { roomId: room.id, itemId: 'chair', x: 5, y: 5 });
  const placed = await waitForEvent(decoSocket, 'FURNITURE_PLACED', 3000);
  expect(placed).toHaveProperty('itemId', 'chair');
  strangerSocket.emit('PLACE_FURNITURE', { roomId: room.id, itemId: 'chair', x: 6, y: 6 });
  const denied = await waitForEvent(strangerSocket, 'PERMISSION_DENIED', 3000);
  expect(denied.action).toBe('PLACE_FURNITURE');
}); });
14. Database Tests – Prisma Integration LINUX File:
__tests__/database/prisma.integration.test.ts
These tests run against the real havenworld_test PostgreSQL instance and verify that the Prisma schema constraints are correctly enforced at the database level. Schema bugs are often invisible in unit tests but surface only in production – this suite catches them early.
// __tests__/database/prisma.integration.test.ts
import { prisma } from '@server/lib/prisma';
import { createTestUser, createTestRoom, createTestItem } from '@tests/helpers/factories';
describe('Prisma Database Integration – Schema Constraints', () => {
  afterAll(async () => {
    await prisma.$disconnect();
  });
  // a) Unique email constraint
  it('should enforce unique email constraint on the User table', async () => {
    await createTestUser({ email: 'unique@test.com' });
    await prisma.user.create({ data: { email: 'unique@test.com', username: 'dup', passwordHash: 'x' } });
    // Prisma wraps the PG unique violation in PrismaClientKnownRequestError
    expect().rejects.toThrow();
  });
  // b) Avatar FK to User + cascade delete
  it('should cascade-delete the Avatar when its parent User is deleted', async () => {
    const user = await createTestUser();
    await prisma.avatar.create({ data: { userId: user.id, bodyColor: '#ff0000', eyeStyle: 'DEFAULT' } });
    await prisma.user.delete({ where: { id: user.id } });
    const avatar = await prisma.avatar.findUnique({ where: { userId: user.id } });
    expect(avatar).toBeNull();
  });
  // c) InventoryItem FK to User and Item
  it('should enforce FK constraints on InventoryItem referencing both User and Item', async () => {
    const user = await createTestUser();
    const item = await createTestItem();
    const inv = await prisma.inventoryItem.create({ data: { userId: user.id, itemId: item.id, quantity: 1 } });
    expect(inv.userId).toBe(user.id);
    expect(inv.itemId).toBe(item.id);
    // FK to non-existent item must fail
    await expect(prisma.inventoryItem.create({ data: { userId: user.id, itemId: 'nonexistent_item', quantity: 1 } })).rejects.toThrow();
  });
  // d) RoomFurniture
  it('should cascade-delete RoomFurniture when the parent Room is deleted', async () => {
    const user = await createTestUser();
    const room = await createTestRoom(user.id);
    const item = await createTestItem();
    const furniture = await prisma.roomFurniture.create({ data: { roomId: room.id, itemId: item.id,

```

```

x: 1, y: 1, placedByUserId: user.id },      });      await
prisma.room.delete({ where: { id: room.id } });      const deleted = await
prisma.roomFurniture.findUnique({ where: { id: furniture.id } });
expect(deleted).toBeNull();      });      //
// e) FishCatch – no cascade
(history preserved on user delete)      //
it('should preserve FishCatch
history records when a User is deleted (no cascade)', async () => {      const
user = await createTestUser();      await prisma.fishCatch.create({ data:
{ userId: user.id, fishType: 'BASS', coinsAwarded: 25 } });      // Expect
the delete to fail because FishCatch has a restrict FK, not cascade      //
(adjust assertion based on your actual schema choice – restrict vs. set null)
await expect(prisma.user.delete({ where: { id:
user.id } })).rejects.toThrow();      // OR if using SET NULL: verify userId
is null after delete      });      //
// f) Migration verification
– all expected tables exist      //
it('should have all 25+
expected tables in the havenworld_test schema after migrations', async () =>
{      const result = await prisma.$queryRaw<{ table_name: string }[]>`
SELECT table_name      FROM information_schema.tables      WHERE
table_schema = 'public'      ORDER BY table_name      `;      const
tableNames = result.map(r => r.table_name);      const expectedTables = [
'User', 'Avatar', 'Room', 'RoomFurniture', 'RoomPermission',      'Item',
'InventoryItem', 'CraftingRecipe', 'CraftingQueue',      'Trade',
'TradeOffer', 'FishCatch', 'InviteCode',      'FriendRequest', 'Friendship',
'Mute', 'Report',      'Session', 'AuditLog',      // ... remaining tables
from your schema      ];      for (const table of expectedTables)
{      expect(tableNames).toContain(table);      }      // Ensure total count
is at least 25
expect(tableNames.length).toBeGreaterThanOrEqual(25);      });      //
// g) Seed verification –
default rooms exist after globalSetup      //
it('should have default
seeded rooms in the database after globalSetup runs', async () => {      const
defaultRooms = await prisma.room.findMany({ where: { isDefault: true } });
expect(defaultRooms.length).toBeGreaterThan(0);      const townSquare =
defaultRooms.find(r => r.slug === 'town_square');
expect(townSquare).not.toBeUndefined();      });      });
15. Load Tests – k6
Scripts LINUX 15.1 Installing k6 on Oracle Cloud VM LINUX # Install k6
from the official repository (Oracle Linux / RHEL-based ARM) sudo dnf install
-y https://dl.k6.io/rpm/repo.rpm sudo dnf install -y k6 # Verify
installation k6 version # Expected: k6 v0.54.x (linux/arm64; go1.23.x; ...)
# Run a quick sanity check against the local server k6 run --vus 1 --duration
5s load-tests/api-auth.js 15.2 WebSocket Concurrent Room Test File: load-
tests/ws-concurrent.js // load-tests/ws-concurrent.js // Scenario: 25
concurrent WebSocket connections, each joining a room and // sending 10 chat
messages. Verifies the Socket.io event loop holds up // under moderate multi-
room load. import ws from 'k6/ws'; import { check, sleep } from 'k6'; import
{ Counter, Trend } from 'k6/metrics'; // Custom metrics for granular
visibility const chatLatency = new Trend('chat_message_latency', true); const
joinLatency = new Trend('room_join_latency', true); const errorsTotal = new
Counter('ws_errors_total'); export const options = { vus: 25, duration:
'60s', thresholds: { // p95 of all WebSocket round-trips must be under
200ms 'chat_message_latency': ['p(95) < 200'], 'room_join_latency':
['p(95) < 300'], // Less than 1% error rate 'ws_errors_total':
['count < 15'], // 1% of 25 VUs * 60s / estimated calls
'ws_session_duration': ['p(95) < 65000'], }, }; const BASE_URL =
'ws://localhost:3000'; const TEST_TOKEN = __ENV.TEST_JWT; // Pass via: k6 run
-e TEST_JWT=... ws-concurrent.js export default function () { const start

```

```

= Date.now();    const res = ws.connect(`${BASE_URL}/socket.io/?
EI0=4&transport=websocket`, {    headers: { Authorization: `Bearer $
{TEST_TOKEN}` },    }, function (socket) {    socket.on('open', () => {
// Join a room immediately after connection    const joinStart =
Date.now();    socket.send(JSON.stringify({ event: 'JOIN_ROOM', data:
{ roomId: 'town_square' } }));    socket.on('message', (rawData) => {
const msg = JSON.parse(rawData);    if (msg.event === 'ROOM_STATE') {
// Measure time from JOIN_ROOM emit to ROOM_STATE receipt
joinLatency.add(Date.now() - joinStart);    // Send 10 chat messages,
spaced 500ms apart    for (let i = 0; i < 10; i++) {    const
chatStart = Date.now();    socket.send(JSON.stringify({
event: 'SEND_CHAT',    data: { roomId: 'town_square', message:
`Load test message ${i} from VU ${__VU}` },    }));
socket.on('message', (raw) => {    const chatMsg =
JSON.parse(raw);    if (chatMsg.event === 'CHAT_MESSAGE') {
chatLatency.add(Date.now() - chatStart);    }    });
sleep(0.5);    }    }    });    socket.on('error', (e) =>
{    errorsTotal.add(1);    console.error(`[VU ${__VU}] WebSocket
error: ${e.error()}`);    });    // Stay connected for the test
duration, then close cleanly    sleep(55);    socket.close();    });
});    check(res, { 'WebSocket connection status is 101': (r) => r &&
r.status === 101 }); } 15.3 HTTP Auth Test File: load-tests/api-
auth.js // load-tests/api-auth.js // Scenario: 50 concurrent POST
/api/auth/login requests. // Verifies the bcrypt comparison and JWT issuance
pipeline holds up // under parallel login bursts (e.g., server restart + mass
reconnect). import http from 'k6/http'; import { check, sleep } from 'k6';
export const options = {    vus: 50,    duration: '30s',    thresholds: {    //
p95 login response time must stay under 500ms even under load
'http_req_duration': ['p(95) < 500'],    // Zero 5xx errors allowed – server
must not crash under login load    'http_req_failed': ['rate <
0.01'],    }, }; const BASE_URL = __ENV.BASE_URL || 'http://localhost:3000';
// Pre-generated test credentials (these users must exist in the test DB)
const CREDENTIALS = [    { email: 'loadtest01@havenworld.local', password:
'LoadTest123!' },    { email: 'loadtest02@havenworld.local', password:
'LoadTest123!' },    // ... up to loadtest50 – seed these in your load test
setup script ]; export default function () {    // Each VU picks its own
credential set to avoid contention    const cred = CREDENTIALS[(__VU - 1) %
CREDENTIALS.length];    const res = http.post(
`${BASE_URL}/api/auth/login`,    JSON.stringify(cred),    { headers:
{ 'Content-Type': 'application/json' }    });    check(res, {    'Login
returns 200': (r) => r.status === 200,    'Response contains accessToken':
(r) => JSON.parse(r.body).accessToken !== undefined,    'No 5xx error': (r)
=> r.status < 500,    });    // Brief pause between requests – simulates
realistic login patterns    sleep(0.5); } 15.4 Concurrent Fishing Economy
Test File: load-tests/fishing-concurrent.js // load-tests/fishing-
concurrent.js // Scenario: 10 users simultaneously catching fish. // After
the test, total coins awarded must equal the sum of individual // catch
rewards – verifies no economy duplication under concurrent load. import ws
from 'k6/ws'; import { check, sleep } from 'k6'; import { SharedArray } from
'k6/data'; import { Counter } from 'k6/metrics'; const totalCoinsAwarded =
new Counter('total_coins_awarded'); const duplicateDetected = new
Counter('duplicate_reward_detected'); // Load pre-generated test tokens for
10 users const testTokens = new SharedArray('tokens', function () {    return
JSON.parse(open('./test-tokens.json')); // Array of 10 JWT strings });
export const options = {    vus: 10,    iterations: 10, // One fish catch per
VU    thresholds: {    // No duplicate rewards detected
'duplicate_reward_detected': ['count == 0'],    }, }; export default function
() {    const token = testTokens[__VU - 1];
ws.connect(`ws://localhost:3000/socket.io/?EI0=4&transport=websocket`,
{    headers: { Authorization: `Bearer ${token}` },    }, function (socket) {

```

```

let caughtOnce = false;    socket.on('open', () =>
{
    socket.send(JSON.stringify({ event: 'JOIN_ROOM', data: { roomId:
'fishing_dock' } }));    });    socket.on('message', (raw) =>
{
    const msg = JSON.parse(raw);    if (msg.event === 'ROOM_STATE') {
// Start fishing immediately after joining
socket.send(JSON.stringify({ event: 'START_FISHING', data: { roomId:
'fishing_dock' } }));    }    if (msg.event === 'FISH_ON')
{
    // Cast with the exact target tension – guaranteed catch
socket.send(JSON.stringify({
    event: 'CAST_COMPLETE',
data: { roomId: 'fishing_dock', tension:
msg.data.tensionTarget },    }));    // Attempt a second cast to
test idempotency    socket.send(JSON.stringify({    event:
'CAST_COMPLETE',    data: { roomId: 'fishing_dock', tension:
msg.data.tensionTarget },    }));    }    if (msg.event ===
'FISH_CAUGHT') {    if (caughtOnce) {    // Second FISH_CAUGHT
from the same session = duplication bug    duplicateDetected.add(1);
}    caughtOnce = true;    totalCoinsAwarded.add(msg.data.reward);
socket.close();    }    });    sleep(30); // Timeout fallback
socket.close();    }); }    15.5 Running k6 and Reading Output    LINUX # Run
the WebSocket concurrent test k6 run \    -e TEST_JWT="$(cat
/home/havenworld/load-test-token.txt)" \    load-tests/ws-concurrent.js # Run
the auth load test k6 run \    -e BASE_URL="http://localhost:3000" \    load-
tests/api-auth.js # Run the fishing economy test (10 VUs, 10 iterations) k6
run load-tests/fishing-concurrent.js # Output to JSON for CI artifact upload
k6 run --out json=results/ws-test-$(date +%Y%m%d).json \    load-tests/ws-
concurrent.js    💡 Reading k6 Output    http_req_duration p(95) – 95th
percentile response time. If this exceeds your threshold, 5% of users are
experiencing slow responses. Optimize first by checking DB query plans, then
Redis hit rate, then bcrypt work factor.    http_req_failed rate – Ratio of
failed requests. Any value above 0 on login or WebSocket connect needs
immediate investigation.    VUs / iterations – k6 shows real-time VU count
and a progress bar. After completion, it prints a pass/fail summary for every
threshold. A red x on any threshold = test failure.    16. CI Integration –
GitHub Actions MAC    16.1 PR Check Workflow    File: .github/workflows/pr-
check.yml    This workflow runs on every pull request to main. It spins up
PostgreSQL 16 and Redis 7 as service containers, runs migrations, executes
the full Jest suite, and fails the PR if coverage falls below 60%.
# .github/workflows/pr-check.yml name: PR Check – Tests & Coverage    on:
pull_request:    branches: [main]    env:    NODE_VERSION: '22'    DATABASE_URL:
postgres://havenworld_user:test_password@localhost:5432/havenworld_test
JWT_SECRET: ci_jwt_secret_not_for_production    JWT_REFRESH_SECRET:
ci_refresh_secret_not_for_production    REDIS_URL: redis://localhost:6379/1
NODE_ENV: test    PORT: 3001    jobs:    test:    name: Jest Test Suite
runs-on: ubuntu-latest    # Service containers start before the job steps
and are torn down after    services:    postgres:    image:
postgres:16    env:    POSTGRES_DB: havenworld_test
POSTGRES_USER: havenworld_user    POSTGRES_PASSWORD: test_password
ports:    - 5432:5432    # Wait until Postgres is actually
accepting connections    options: >-    --health-cmd pg_isready
--health-interval 10s    --health-timeout 5s    --health-
retries 5    redis:    image: redis:7-alpine    ports:
- 6379:6379    options: >-    --health-cmd "redis-cli ping"
--health-interval 10s    --health-timeout 5s    --health-
retries 5    steps:    # — 1. Checkout repository
- name: Checkout code    uses:
actions/checkout@v4    # — 2. Set up Node.js 22 with npm cache
- name: Set up Node.js ${{ env.NODE_VERSION }}
uses: actions/setup-node@v4    with:    node-version: $
{{ env.NODE_VERSION }}    cache: 'npm'    # — 3. Install
dependencies    - name: Install npm

```

```

dependencies          run: npm ci          # — 4. Generate Prisma client
                                - name: Generate Prisma client
run: npx prisma generate          # — 5. Run database migrations on
havenworld_test          - name: Run Prisma migrations on test
database          run: npx prisma migrate deploy --schema=prisma/schema.prisma
# — 6. Build TypeScript (catch compile errors before tests) —          - name:
TypeScript build check          run: npx tsc --noEmit --project
tsconfig.test.json          # — 7. Run Jest with coverage
                                - name: Run Jest test suite with
coverage          run: npm run test:coverage          # Jest exits with code 1
if coverage thresholds not met,          # which automatically fails this step
and the PR.          # — 8. Upload coverage report as artifact
                                - name: Upload coverage report          uses:
actions/upload-artifact@v4          if: always() # Upload even if tests failed
(for debugging)          with:          name: coverage-report-$
{{ github.sha }}          path: coverage/          retention-days: 14
# — 9. Post coverage summary to PR comment —          - name:
Comment coverage summary on PR          uses: davelosert/vitest-coverage-
report-action@v2          if: always()          with:          json-summary-
path: coverage/coverage-summary.json          16.2 Nightly Load Test Workflow File:
.github/workflows/load-test.yml          # .github/workflows/load-test.yml name:
Nightly Load Tests on:          schedule:          # Run every night at 02:00 UTC (21:00
CDT) — off-peak hours          - cron: '0 2 * * *'          workflow_dispatch: # Allow
manual trigger from GitHub UI          jobs:          load-test:          name: k6 Load Tests on
Oracle VM          runs-on: ubuntu-latest          steps:          # — 1. Checkout for
script access          - name: Checkout code
uses: actions/checkout@v4          # — 2. Copy k6 scripts to Oracle VM via SCP
                                - name: Copy load test scripts to Oracle VM
uses: appleboy/scp-action@v0.1.7          with:          host: $
{{ secrets.ORACLE_VM_HOST }}          username: $
{{ secrets.ORACLE_VM_USER }}          key: ${{ secrets.ORACLE_VM_SSH_KEY }}
source: "load-tests/"          target: "/home/havenworld/load-tests/"
# — 3. SSH into Oracle VM and run k6 scripts —          - name:
Run k6 load tests on Oracle VM          uses: appleboy/ssh-action@v1.0.3
with:          host: ${{ secrets.ORACLE_VM_HOST }}          username: $
{{ secrets.ORACLE_VM_USER }}          key: ${{ secrets.ORACLE_VM_SSH_KEY }}
script: |          cd /home/havenworld          mkdir -p load-test-
results          echo "Running WebSocket concurrent test..."
k6 run \          -e TEST_JWT="$(cat /home/havenworld/.load-test-token)"
\          --out json=load-test-results/ws-$(date +%Y%m%d).json \
load-tests/ws-concurrent.js          echo "Running Auth API load test..."
k6 run \          -e BASE_URL="http://localhost:3000" \          --
out json=load-test-results/auth-$(date +%Y%m%d).json \          load-
tests/api-auth.js          echo "Running Fishing economy test..."
k6 run \          --out json=load-test-results/fishing-$(date +%Y%m
%d).json \          load-tests/fishing-concurrent.js          echo
"All load tests complete."          # — 4. Download results from Oracle VM
                                - name: Download load test results
uses: appleboy/scp-action@v0.1.7          with:          host: $
{{ secrets.ORACLE_VM_HOST }}          username: $
{{ secrets.ORACLE_VM_USER }}          key: ${{ secrets.ORACLE_VM_SSH_KEY }}
source: "/home/havenworld/load-test-results/*.json"          target: "load-
test-results/"          # — 5. Upload results as GitHub Actions artifact
                                - name: Upload k6 results          uses: actions/upload-
artifact@v4          with:          name: k6-results-{{ github.run_id }}
path: load-test-results/          retention-days: 30          # — 6. Email
results via Resend if thresholds breached —          - name: Send load
test summary email          if: failure()          run: |          curl -X
POST https://api.resend.com/emails \          -H "Authorization: Bearer $
{{ secrets.RESEND_API_KEY }}" \          -H "Content-Type:

```

```

application/json" \
-noreply@havenworld.gg",
"subject": "🚨 HavenWorld Nightly Load Test FAILED - ${github.run_id}",
"text": "One or more k6 threshold checks failed. Review artifacts at $
${github.server_url}/${github.repository}/actions/runs/${github.run_id}"
}'
🚨 Important: GitHub Secrets Setup
Before the load test workflow runs, configure these secrets in your GitHub
repository settings under Settings → Secrets and variables → Actions:
ORACLE_VM_HOST – Public IP of your Oracle Cloud VM
ORACLE_VM_USER – SSH username (e.g., havenworld)
ORACLE_VM_SSH_KEY – Private SSH key (the full PEM contents, not the path)
RESEND_API_KEY – Your Resend API key for email alerts
17. Test Utilities & Helpers
17.1 factories.ts – Test Data Factories
File: __tests__/helpers/factories.ts // __tests__/helpers/factories.ts
// Factory functions create realistic, minimal test data with sensible defaults.
// All accept Partial overrides so tests only specify what they care about.
import { prisma } from '@server/lib/prisma';
import { hashPassword } from '@server/utils/password';
import { User, Room, Item, InviteCode } from '@prisma/client';
import { randomUUID } from 'crypto';

export async function createTestUser(overrides: Partial<{ email: string; username: string; password: string; emailVerified: boolean; isBanned: boolean; isAdmin: boolean; havenCoins: number; weeklyEarnings: number; weeklyEarningsCap: number; lastLoginClaimDate: Date | null; verificationToken: string; }> = {}): Promise<User> {
  const uniqueSuffix = randomUUID().slice(0, 8);
  const passwordHash = await hashPassword(overrides.password || 'SecurePass1!');
  return prisma.user.create({
    data: {
      email: overrides.email || `testuser_${uniqueSuffix}@test.com`,
      username: overrides.username || `testuser_${uniqueSuffix}`,
      passwordHash,
      emailVerified: overrides.emailVerified ?? true, // Default to verified for most tests
      isBanned: overrides.isBanned ?? false,
      isAdmin: overrides.isAdmin ?? false,
      havenCoins: overrides.havenCoins ?? 1000,
      weeklyEarnings: overrides.weeklyEarnings ?? 0,
      weeklyEarningsCap: overrides.weeklyEarningsCap ?? 10000,
      lastLoginClaimDate: overrides.lastLoginClaimDate ?? null,
      verificationToken: overrides.verificationToken ?? null,
    },
  });
}

export async function createTestRoom(ownerId: string, overrides: Partial<{ name: string; slug: string; privacy: 'PUBLIC' | 'FRIENDS_ONLY' | 'PASSWORD' | 'LOCKED'; password: string | null; capacity: number; isDefault: boolean; type: string; }> = {}): Promise<Room> {
  const uniqueSuffix = randomUUID().slice(0, 8);
  return prisma.room.create({
    data: {
      name: overrides.name || `Test Room ${uniqueSuffix}`,
      slug: overrides.slug || `test_room_${uniqueSuffix}`,
      ownerId,
      privacy: overrides.privacy || 'PUBLIC',
      password: overrides.password ?? null,
      capacity: overrides.capacity ?? 20,
      isDefault: overrides.isDefault ?? false,
      type: overrides.type ?? 'LOFT',
    },
  });
}

export async function createTestItem(overrides: Partial<{ name: string; type: string; havenCoinCost: number; }> = {}): Promise<Item> {
  const uniqueSuffix = randomUUID().slice(0, 8);
  return prisma.item.create({
    data: {
      name: overrides.name || `Test Item ${uniqueSuffix}`,
      type: overrides.type || 'FURNITURE',
      havenCoinCost: overrides.havenCoinCost ?? 100,
    },
  });
}

export async function createTestInviteCode(overrides: Partial<{ code: string; used: boolean; createdByUserId: string | null; }> = {}): Promise<InviteCode> {
  return prisma.inviteCode.create({
    data: {
      code: overrides.code || `TESTCODE_${randomUUID().slice(0, 8).toUpperCase()}`,
      used: overrides.used ?? false,
      createdByUserId: overrides.createdByUserId ??

```

```

null,    },    }); } 17.2 socketHelpers.ts – Socket.io Test Utilities
File: __tests__/helpers/socketHelpers.ts //
__tests__/helpers/socketHelpers.ts import { io as ClientIO, Socket } from
'socket.io-client'; import { generateTestToken } from './jwtHelpers'; // —
createAuthenticatedSocket _____ // Creates a
socket.io-client connection pre-authenticated with a test JWT. // Adds the
socket to the sockets array so afterEach can close it. export async function
createAuthenticatedSocket(  serverUrl: string,  userId: string,  sockets:
Socket[] ): Promise<Socket> {  const token = generateTestToken(userId);
return new Promise((resolve, reject) => {  const socket =
ClientIO(serverUrl, {  auth: { token },  transports: ['websocket'],
// Skip polling – tests need deterministic transport  }));
socket.on('connect', () => {  sockets.push(socket);
resolve(socket);  });  socket.on('connect_error', (err) =>
{  reject(new Error(`Socket connection failed: $
{err.message}`));  });  // Timeout if connection takes more than 5
seconds  setTimeout(() => reject(new Error('Socket connection timeout')),
5000);  }); } // — waitForEvent
_____ // Returns a Promise that
resolves with the first event payload matching eventName. // Rejects if the
event does not arrive within the timeout (default: 5000ms). // Use this
instead of nested callbacks to keep test code readable. export function
waitForEvent(  socket: Socket,  eventName: string,  timeoutMs = 5000 ):
Promise<any> {  return new Promise((resolve, reject) => {  const timer =
setTimeout(() => {  socket.off(eventName, handler);  reject(new
Error(`Timed out waiting for event "${eventName}" after ${timeoutMs}ms`));
}, timeoutMs);  function handler(data: any) {  clearTimeout(timer);
socket.off(eventName, handler);  resolve(data);  }
socket.on(eventName, handler);  }); } // — closeAllSockets
_____ // Gracefully disconnects all
sockets in the array and clears it. // Call this in afterEach to prevent open
handle warnings from Jest. export async function closeAllSockets(sockets:
Socket[]): Promise<void> {  await Promise.all(  sockets.map(socket => new
Promise<void>(resolve => {  if (socket.connected)
{  socket.once('disconnect', () => resolve());
socket.disconnect();  } else {  resolve();  }  })))  );
sockets.length = 0; // Mutate the array in-place to clear it } 17.3
dbHelpers.ts – Database Utilities File: __tests__/helpers/dbHelpers.ts //
__tests__/helpers/dbHelpers.ts import { prisma } from
'@server/lib/prisma'; // — truncateAllTables
_____ // TRUNCATES all application tables
in the correct FK-safe order. // Uses RESTART IDENTITY to reset auto-
increment sequences. // Uses CASCADE to handle FK dependencies automatically.
// NEVER drops tables – schema and migrations are preserved. export async
function truncateAllTables(): Promise<void> {  const dbUrl =
process.env.DATABASE_URL;  if (!dbUrl || !dbUrl.includes('_test'))
{  throw new Error('SAFETY: truncateAllTables must only run against
havenworld_test');  }  // Disable FK checks temporarily, truncate
everything, re-enable.  // Order matters: child tables before parent tables
if not using CASCADE.  await prisma.$executeRawUnsafe(`  TRUNCATE TABLE
"FishCatch",  "RoomFurniture",  "RoomPermission",
"CraftingQueue",  "TradeOffer",  "Trade",  "InventoryItem",
"AuditLog",  "Session",  "FriendRequest",  "Friendship",
"Mute",  "Report",  "InviteCode",  "Avatar",  "Room",
"Item",  "CraftingRecipe",  "User"  RESTART IDENTITY CASCADE;
`); } // — seedMinimalData _____ //
Seeds only the data required for the test suite to function: // default rooms
and the base item catalog. Does NOT seed users. export async function
seedMinimalData(): Promise<void> {  // Create default rooms (isDefault rooms
cannot be deleted by users)  await prisma.room.createMany({  data: [

```

```
{ name: 'Town Square', slug: 'town_square', privacy: 'PUBLIC', capacity: 50,
isDefault: true, type: 'PUBLIC_SPACE' },      { name: 'Fishing Dock', slug:
'fishing_dock', privacy: 'PUBLIC', capacity: 20, isDefault: true, type:
'FISHING_DOCK' },      { name: 'Workshop', slug: 'workshop', privacy:
'PUBLIC', capacity: 15, isDefault: true, type: 'WORKSHOP' },      ],
skipDuplicates: true, // Safe to call multiple times without
error  }); // Create a minimal item catalog for inventory/economy tests
await prisma.item.createMany({    data: [      { name: 'Basic Chair', type:
'FURNITURE', havenCoinCost: 100 },      { name: 'Cozy Lamp', type:
'FURNITURE', havenCoinCost: 150 },      { name: 'HavenWorld Hoodie', type:
'CLOTHING', havenCoinCost: 500 },    ],    skipDuplicates: true,  }); }
17.4 jwtHelpers.ts – JWT Test Utilities  File:
```

```
__tests__/helpers/jwtHelpers.ts  // __tests__/helpers/jwtHelpers.ts import
jwt from 'jsonwebtoken'; const TEST_SECRET = process.env.JWT_SECRET ||
'test_jwt_secret_not_for_production_use_only'; // — generateTestToken
// Generates a valid, signed JWT for
a given userId. // Default expiry is 15 minutes – enough for any test to
complete. export function generateTestToken(  userId: string,  expiresIn:
string | number = '15m' ): string {  return jwt.sign(    {    userId,
iat: Math.floor(Date.now() / 1000),    },    TEST_SECRET,    { expiresIn }
); } // — generateExpiredToken //
Generates a JWT that expired 1 second ago. // Used to test middleware
rejection of expired tokens. export function generateExpiredToken(userId:
string): string {  return jwt.sign(    { userId },    TEST_SECRET,
{ expiresIn: -1 } // Expired 1 second in the past  ); } // —
generateTokenWithWrongSecret // Generates a
structurally valid JWT signed with a different secret. // Used to test that
the middleware rejects tokens from other issuers. export function
generateTokenWithWrongSecret(userId: string): string {  return jwt.sign(
{ userId },    'this_is_definitely_the_wrong_secret',    { expiresIn: '15m'
}  ); } // — decodeTestToken //
```

Decodes (does not verify) a JWT payload. Useful for asserting // that a newly
issued token contains the expected claims. export function

```
decodeTestToken(token: string): jwt.JwtPayload | null {  return
jwt.decode(token) as jwt.JwtPayload | null; } 18. Test Coverage
```

Checklist The table below lists every service and feature tested in Part 7A,
the test type(s) applied, and the exact file path. Use this as a PR review
checklist and a living document – add rows as new services are built in Parts
8–10.

File Path	Cases	Service / Feature	Status	Test Type
Auth Middleware	Unit			
src/server/middleware/__tests__/auth.test.ts	6		✓	Unit
Complete		ModerationService		
src/server/services/__tests__/moderation.test.ts	8		✓	
Complete		RoomManager		Unit
src/server/services/__tests__/roomManager.test.ts	8			
✓ Complete		InventoryService		Unit
src/server/services/__tests__/inventoryService.test.ts	8			
✓ Complete		WorkshopService / CraftingQueue		
Unit		src/server/services/__tests__/workshopService.test.ts		
5	✓ Complete	TradeManager (P2P)		
Unit		src/server/services/__tests__/tradeManager.test.ts		
8	✓ Complete	Auth HTTP Routes		
Integration (HTTP)		__tests__/integration/auth.integration.test.ts		
10	✓ Complete	Socket.io Connection		
& Room Flow		Integration (WebSocket)		
__tests__/integration/socketio.integration.test.ts	8			
✓ Complete		Fishing Mini-Game Server Logic		
Integration (WebSocket)				
__tests__/integration/fishing.integration.test.ts	6			

✓ Complete Loft Privacy & Doorbell
 Integration (WebSocket)
 __tests__/integration/loft.integration.test.ts 7 ✓
 Complete Prisma Schema Constraints
 Integration (Database)
 __tests__/database/prisma.integration.test.ts 7 ✓
 Complete WebSocket Room Load (25 VUs)
 Load (k6) load-tests/ws-concurrent.js Threshold: p95
 <200ms, errors <1% ✓ Complete Auth
 API Load (50 VUs) Load (k6) load-tests/api-auth.js
 Threshold: p95 <500ms, 0 5xx ✓ Complete
 Fishing Economy Concurrency (10 VUs) Load (k6) load-
 tests/fishing-concurrent.js Threshold: 0 duplicate rewards
 ✓ Complete Test Factories (User, Room, Item,
 Code) Helper __tests__/helpers/factories.ts
 4 factories ✓ Complete Socket
 Helpers (create, wait, close) Helper
 __tests__/helpers/socketHelpers.ts 3 utilities ✓
 Complete DB Helpers (truncate, seed)
 Helper __tests__/helpers/dbHelpers.ts 2 utilities
 ✓ Complete JWT Helpers (generate, expire,
 wrong secret) Helper __tests__/helpers/jwtHelpers.ts
 4 utilities ✓ Complete GitHub
 Actions PR Check CI .github/workflows/pr-check.yml
 Postgres 16 + Redis 7 services, coverage gate ≥60% ✓ Complete
 GitHub Actions Nightly Load Test CI
 .github/workflows/load-test.yml SSH to Oracle VM, run all k6
 scripts, email on failure ✓ Complete Total test
 cases authored in Part 7A: 81 functional tests + 14 thresholds across 3 load
 scripts. 💡 Next Steps – Part 7B Preview Part 7B will cover client-
 side testing: Vitest + React Testing Library for the Pixi.js game canvas
 components, mock Socket.io in the browser, and Playwright end-to-end tests
 that drive a real Chromium browser through the full login → room join → chat
 flow against the local dev server. HavenWorld Build Guide Part 7A –
 Server-Side Testing | Daniel Boone | 18 September 2026 |
 github.com/dboone323/HavenWorld | GPL-3.0 License